

FATAL FORCE

DATA SCIENCE REPORT



by Kadir Arslan

Created: 20.06.2025

Last Edited: 18.04.2026

Fatal Force & Socioeconomic Patterns Report

This report addresses one of the most sensitive and highly debated issues in the United States: the use of deadly force by police officers. Using data collected by The Washington Post between 2015 and 2017, we analyze thousands of deaths related to police encounters as well as numerous socioeconomic indicators from U.S. Census Bureau datasets. By combining information on poverty rates, median household income, high school graduation rates, ethnic composition, age, and gender, this project aims to uncover broader societal patterns that may lie behind the statistics.

As part of our analysis, we compare trends across states and cities to better understand how economic and demographic conditions are linked to fatal police-related incidents. While data alone cannot fully explain the complex relationships underlying these incidents, it can highlight significant disparities, raise relevant questions, and provide valuable insights into the social context in which police violence occurs in the United States.

This project was designed not only as a technical data analysis using Python, Pandas, and Plotly, but also as an opportunity to explore how data science can help visualize and better understand real-world social issues.

Source Data Files (CSV Format)

GitHub links for the Data Files: github.com

Imported Libraries

```
import numpy as np
import pandas as pd
import plotly.express as px
import matplotlib.pyplot as plt
import seaborn as sns
from pandas.plotting import register_matplotlib_converters
from matplotlib.ticker import PercentFormatter
from collections import Counter
from census import Census
from us import states
```

Display Formatting for Financial Data

To improve readability and ensure consistent formatting, we are going to configure pandas to display floating-point numbers and values in a more appropriate way.

```
# Notebook Presentation:
pd.options.display.float_format = '{:,.2f}'.format
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
```

Acquiring the Source and Exploring Initial Dataset

```
df_income = pd.read_csv('.../Median_Household_Income_2015.csv', encoding="windows-1252")
df_poverty = pd.read_csv('.../Pct_People_Below_Poverty_Level.csv', encoding="windows-1252")
df_high_school = pd.read_csv('.../Pct_Over_25_Completed_High_School.csv', encoding="windows-1252")
df_race_shares = pd.read_csv('.../Share_of_Race_By_City.csv', encoding="windows-1252")
df_fatalities = pd.read_csv('.../Deaths_by_Police_US.csv', encoding="windows-1252")
```

We import the data files with using the *pandas.read_csv()* function, which converts the raw CSV files into structured DataFrames for further processing and analysis.

The parameter *encoding = "windows – 1252"* was specifically used to correctly interpret special characters and text formatting, as commonly found in CSV files, thereby preventing character corruption or decoding errors during the import process.

Obtaining the Structure of the Dataset

Since the project involves multiple datasets, we are going to create a reusable function to efficiently validate each DataFrame and summarize the results in a consistent and optimized way.

Defining a Reusable Data Overview Function

We are going to create a function called *display_overview()* to provide a quick diagnostic overview of a dataset by displaying its dimensions, checking for missing or duplicate entries, and providing structural information about the DataFrame.

In addition, the function improves the data cleaning workflow by automatically removing problematic entries and displaying five random samples, enabling a quick and practical understanding of the dataset before beginning a more in-depth analysis.

```
def display_overview(df):
    print(f"Number of Rows (Entries): {df.shape[0]}")
    print(f"Number of Columns (Features): {df.shape[1]}")

    print("\nMissing and Duplicated Entries:")

    if df.isna().values.any():
        na_sum = df.isna().values.any().sum()
        print(f"The Amount of Entries with Missing Values: {na_sum}")
        df = df.dropna()
        print("These missing entries were removed.")
    else:
        print("There is no entry with missing values.")

    if df.duplicated().values.any():
        dup_sum = df.duplicated().values.any().sum()
        print(f"The Amount of Entries with Duplicated Values {dup_sum}")
        df = df.drop_duplicates()
        print("These duplicated entries were removed.")
    else:
        print("There is no entry with duplicated values.")

    print("\nOverview of the DataFrame:")
    print(df.info())

    print("\nRandom five Entries from the DataFrame:")
    print(df.sample(5))
```

Applying the Function:

We are going to apply the same function to all DataFrames.

```

print("\nFatality DataFrame")
display_overview(df_fatalities)

print("\nIncome DataFrame:")
display_overview(df_income)

print("\nPoverty DataFrame:")
display_overview(df_poverty)

print("\nHigh School Graduation DataFrame:")
display_overview(df_high_school)

print("\nRace Demography DataFrame:")
display_overview(df_race_shares)

```

Fatality DataFrame:

Number of Rows (Entries): 2535
 Number of Columns (Features): 14

Missing and Duplicated Entries:

The Amount of Entries with Missing Values: 1
 These missing entries were removed.
 There is no entry with duplicated values.

Overview of the DataFrame:

Data columns (total 14 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	id	2254 non-null	int64
1	name	2254 non-null	str
2	date	2254 non-null	str
3	manner_of_death	2254 non-null	str
4	armed	2254 non-null	str
5	age	2254 non-null	float64
6	gender	2254 non-null	str
7	race	2254 non-null	str
8	city	2254 non-null	str

```

9    state                2254 non-null    str
10   signs_of_mental_illness 2254 non-null    bool
11   threat_level           2254 non-null    str
12   flee                  2254 non-null    str
13   body_camera            2254 non-null    bool
dtypes: bool(2), float64(1), int64(1), str(10)

```

Random five Entries from the DataFrame:

id	name	date	manner_of_death	armed	age	gender	race	city	state	signs_of_mental_illness	threat_level	flee	body_camera
2321	Alonzo E. Ashley	11/02/17	shot	gun	27.00	M	B	Kansas City	MO	False	attack	Not fleeing	False
719	Shamir Terrel Palmer	08/08/15	shot	gun	24.00	M	B	Summerville	SC	False	attack	Car	False
1736	Dalvin Hollins	27/07/16	shot	unarmed	19.00	M	B	Tempe	AZ	True	attack	Foot	False
1839	Ed Holup	01/09/16	shot	vehicle	60.00	M	W	Sylvania Township	OH	True	other	Not fleeing	False
1764	Earl Pinckney	07/08/16	shot	knife	20.00	M	B	Harrisburg	PA	True	other	Not fleeing	False

The Fatality DataFrame consists of 2535 entries along 14 different features and contains detailed information on fatal shootings by police officers in the United States, including demographic characteristics such as age, gender, and race, as well as situational factors such as the level of threat, signs of mental illness, the use of body cameras, and whether the individual attempted to flee.

After removing entries with missing values, the dataset provides a relatively clean and meaningful basis for examining patterns and correlations between incidents involving police officers and general socioeconomic conditions in various places.

The columns “date”, “race” and “gender” requires data type or format conversions.

Income DataFrame:

Number of Rows (Entries): 29322

Number of Columns (Features): 3

Missing and Duplicated Entries:

The Amount of Entries with Missing Values: 1

These missing entries were removed.

There is no entry with duplicated values.

Overview of the DataFrame:

Data columns (total 3 columns):

```
#   Column          Non-Null Count  Dtype
---  -
0   Geographic Area  29271 non-null    str
1   City             29271 non-null    str
2   Median Income    29271 non-null    str
dtypes: str(3)
```

Random five Entries from the DataFrame:

	Geographic Area	City	Median Income
3039	CA	Rancho Palos Verdes city	118780
8791	IA	Ossian city	51583
17240	NY	Granville village	38194
26533	UT	South Ogden city	55605
0	AL	Abanda CDP	11207

The Income DataFrame contains information on the median household income of 29,322 cities and geographic areas in the United States in 2015, thereby providing a comprehensive socioeconomic perspective that can later be compared with patterns of fatal police interventions.

Even from the first few entries, significant differences in income levels between different regions are evident, illustrating the great economic diversity within American communities.

The column “Median Income” required datatype conversion from string to numeric.

Poverty DataFrame:

Number of Rows (Entries): 29329

Number of Columns (Features): 3

Missing and Duplicated Entries:

There is no entry with missing values.

There is no entry with duplicated values.

Overview of the DataFrame:

Data columns (total 3 columns):

```
#   Column          Non-Null Count  Dtype
---  -
0   Geographic Area  29329 non-null    str
1   City             29329 non-null    str
2   poverty_rate     29329 non-null    str
dtypes: str(3)
```

Random five Entries from the DataFrame:

	Geographic Area	City	poverty_rate
25150	TX	Harker Heights city	17.1
5956	ID	Melba city	16.4
206	AL	Gilbertown town	21.5
1741	AR	Mount Ida city	16
351	AL	Mobile city	23.3

The Poverty DataFrame contains information on poverty rates for 29,329 cities and geographic areas (States) in the United States in 2015, making it a key resource for studying socioeconomic conditions in various regions.

Even a cursory glance at the sample values reveals significant differences in poverty levels across communities, suggesting that economic inequality varies considerably from state to state and may play a significant role in broader societal trends.

The column “poverty_rate” requires datatype conversion from string to numeric type.

High School Graduation DataFrame:

Number of Rows (Entries): 29329

Number of Columns (Features): 3

Missing and Duplicated Entries:

There is no entry with missing values.

There is no entry with duplicated values.

Overview of the DataFrame:

Data columns (total 3 columns):

#	Column	Non-Null Count	Dtype
0	Geographic Area	29329 non-null	str
1	City	29329 non-null	str
2	percent_completed_hs	29329 non-null	str

dtypes: str(3)

Random five Entries from the DataFrame:

	Geographic Area	City	percent_completed_hs
7691	IN	Hope town	92
9531	KS	Montezuma city	59
19367	OH	Champion Heights CDP	91.3
12032	MI	K. I. Sawyer CDP	92.6
26905	VA	Fairfax city	92.4

The High-school Graduation DataFrame contains the percentage of adults aged 25 and older who had a high school diploma in 2015 across 29,329 cities. These individuals may have earned their high school diploma in any state and at any time; the metric reported is the proportion of people with a high school diploma in the respective cities.

Even within the small sample shown above, there are already clear differences in educational attainment between communities, making this dataset particularly valuable for examining the extent to which education-related socioeconomic conditions might be linked to broader social patterns and statistics on fatal police interventions.

The column “percent_completed_hs” requires datatype conversion from string to numeric type.

Racial Demography DataFrame:

Number of Rows (Entries): 29268

Number of Columns (Features): 7

Missing and Duplicated Entries:

There is no entry with missing values.

There is no entry with duplicated values.

Overview of the DataFrame:

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	Geographic Area	29268 non-null	str
1	City	29268 non-null	str
2	share_white	29268 non-null	str
3	share_black	29268 non-null	str
4	share_native_american	29268 non-null	str
5	share_asian	29268 non-null	str
6	share_hispanic	29268 non-null	str

dtypes: str(7)

Random five Entries from the DataFrame:

Geographic Area	City	share_white	share_black	share_native_american	share_asian	share_hispanic
MN	Waverly city	97.3	0.5	0.4	0.8	1.3
SC	Centerville CDP	83.3	13.3	0.2	1.1	2.7
NJ	New Egypt CDP	91.4	2	0.8	1	10.5
CT	Mansfield Center CDP	86.3	2	0.7	7.2	3
NM	Acomita Lake CDP	3.4	0	93.3	0	3.1

The Racial Demography DataFrame contains information on the proportion of various ethnic groups in 29,268 cities and geographic areas (states) across the United States in 2015, including the percentage distribution of the White, Black, Native American, Asian, and Hispanic populations.

Even on the randomly selected entries, clear demographic differences between communities can be observed, which highlights the diversity of the United States and provides an important basis for analyzing the extent to which ethnic composition might be linked to regional patterns in the data on fatal police interventions.

It is also noteworthy to state that, a significant portion of “Hispanic” people are categorized as both white and Hispanic at the Census’ US demography data, so total shares can exceed 100 at corresponding cities.

The columns for ethnic shares require datatype conversions from string to numeric.

Performing Data Type and Format Transformations

As observed in the overview, some data features are not entered or exported in desired datatype in all DataFrames. We are going to execute some datatype conversions to prepare the datasets for analysis.

Standardization of the “Fatality DataFrame”

The Fatality DataFrame is structured slightly differently from the other four data frames. We will perform some specific and custom data conversions on this data frame.

Date Data Conversion:

The date column was originally stored as a string, which limits the possibilities for chronological and time-based analyses.

By converting the column to a datetime object using `pd.to_datetime()`, the dataset is now suitable for operations such as extracting years and months, identifying trends over time, and creating time-based visualizations more efficiently.

```
df_fatalities["date"] = pd.to_datetime(df_fatalities["date"], format="mixed")
```

Gender Data Conversions:

In the original dataset, gender information was represented by short categorical codes such as “M” and “F.”

To improve readability and make the results easier to present in visualizations and analyses, we define a mapping dictionary and apply it to the DataFrame to convert these abbreviations into their full terms.

```
# Defining the mapping:
gender_mapping = {"M": "Male", "F": "Female"}

# Applying the mapping
df_fatalities.gender = df_fatalities.gender.map(gender_mapping)
```

Race Data Conversions:

In the race column, demographic categories were originally stored using abbreviations such as “W,” “B,” and “H,” which, are less suitable for reporting.

Therefore, we are going to create a custom mapping structure to convert these short codes into fully readable ethnic categories, making the dataset significantly clearer for analysis, visualization, and interpretation.

```
race_mapping = {
    "W": "White",
    "B": "Black",
    "H": "Hispanic",
    "A": "Asian",
    "N": "Native American",
    "O": "Other"
}
# Applying the mapping
df_fatalities.race = df_fatalities.race.map(race_mapping)
```

State Data Conversions:

The Fatality DataFrame originally listed U.S. states using two-letter abbreviations such as “CA” and “TX,” which can impair readability in large-scale visualizations.

To improve clarity, we are going to use a mapping dictionary to generate a new column containing the full names of the states. The `.pop()` and `.insert()` methods are then applied to insert this new column at a specific location within the DataFrame structure, thereby improving organization and making it easier to review the dataset.

```
state_mapping = {
    'AL': 'Alabama', 'AK': 'Alaska', 'AZ': 'Arizona', 'AR': 'Arkansas',
    'CA': 'California', 'CO': 'Colorado', ..., ..., ...
}
# Applying the mapping and defining exact place for it:
df_fatalities["State_Full"] = df_fatalities["state"].map(state_mapping)
col = df_fatalities.pop("State_Full")
df_fatalities.insert(10, "State_Full", col)
```

Standardization of the Remaining DataFrames

The remaining four datasets come from the U.S. Census Bureau and have a similar structures as we observed earlier. Therefore, we are going to define functions to perform data transformations upon them as together. Then, we are going to apply these functions to the DataFrames simultaneously to save time and reduce the amount of coding required.

Adding Full State Names to Datasets

To improve the consistency and readability of all datasets, we are going to define a new function to add the full names of the states to the DataFrames based on census data, in addition to the existing state abbreviations. This step makes subsequent merging, filtering, grouping, and visualization processes significantly more intuitive, particularly for state-level comparisons and geographic analyses.

We also change the name of the column “Geographic Area” to the “State” in these 4 DataFrames to have a clearer view throughout the analysis.

```
def add_full_state(df):  
    df.rename(columns={'Geographic Area': 'State'})  
    df["State_Full"] = df["State"].map(state_mapping)  
    col = df.pop("State_Full")  
    df.insert(1, "State_Full", col)
```

Applying the function:

```
add_full_state(df_income)  
add_full_state(df_poverty)  
add_full_state(df_high_school)  
add_full_state(df_race_shares)
```

Converting String-Based Numerical Values into Numeric Datatypes

Several columns in the datasets contain numerical information that was stored as string values, which prevents mathematical operations, statistical calculations, and visual analyses from being performed correctly.

To resolve this issue, we are going to define a reusable conversion function that cleans up formatting inconsistencies—such as commas and placeholder symbols—

before converting the columns to the correct numeric data types using `pd.to_numeric()`.

```
# Numeric data conversions for the DataFrames
def convert_str_to_numeric(df, columns=None):
    if columns is None:
        columns = []
    for column in columns:
        df[column] = pd.to_numeric(df[column].str.replace(',', ''),
regex=False).replace('-', '0', regex=False), errors='coerce')
```

Applying the function:

```
convert_str_to_numeric(df_income, ["Median Income"])
convert_str_to_numeric(df_poverty, ["poverty_rate"])
convert_str_to_numeric(df_high_school, ["percent_completed_hs"])
convert_str_to_numeric(df_race_shares, ["share_white", "share_black",
"share_native_american", "share_asian", "share_hispanic"])
```

Retrieving and Integrating Population Data through Census API Interface

As observed during the initial review of the datasets, many key socioeconomic indicators—such as median household income, poverty rates, and educational attainment—were originally collected at the city level.

Since these datasets contain nearly 30,000 entries and cover a very large proportion of cities in the United States, conducting a more comprehensive analysis at the state level requires converting these city-level metrics into representative values for the entire state.

However, calculating simple arithmetic averages would lead to misleading results, as cities vary enormously in population; giving a small town the same weight as a large metropolitan area would skew the final statistics.

For this reason, population-weighted averages were used—a method in which each city is weighted in proportion to its population, ensuring that larger metropolitan areas have a more realistic impact on the final state-level metrics.

To perform this calculation correctly, we need population data each city and we are going to retrieve this data via an API integration using the United States Census Bureau's API.

Initializing Census:

To calculate accurate statistics, we are going to retrieve population data for every city and town in the United States directly via the United States Census Bureau's API using the Python package “*census*.”

Here, we initialize access to Census using the API key we obtained “for free” from the Census website.

```
c = Census("31b41...f")
```

Getting Population Values for Every City:

The query is specifically targeted the total population variable (*B01003_001E*) from the 2015 American Community Survey (ACS5), and the returned JSON-like data was converted into a DataFrame to enable efficient preprocessing, transformation, and merging with the existing datasets.

```
# Pulling Total Population (B01003_001E) and Name for all "places" (cities/towns)
population_data = c.acs5.state_place(('NAME', 'B01003_001E'),
                                     Census.ALL, Census.ALL,
                                     year=2015)

# Converting population data to a dataframe so that we can perform merging
df_pop = pd.DataFrame(population_data)
```

Defining Function to add population to each city:

To correctly add population figures to each city-level data record, we are going to construct a data integration function that utilizes various key techniques from data preprocessing and feature engineering.

Since city names in the various datasets were entered in slightly different formats (for example, with suffixes such as “city,” “CDP,” “town,” or “village”), text cleaning based on regular expressions is applied to standardize naming conventions and improve matching accuracy between the Census API data and the DataFrames.

The API's combined location field is then split into separate columns for city and state, enabling a multi-column merge that used both standardized city names and full state names as matching keys.

Finally, redundant intermediate columns that had been created during preprocessing are removed, resulting in a clearer, analysis-ready DataFrame structure that now contains population figures for each corresponding city entry.

```
def add_population(df):
    # We will use "City name, State name" to retract exact populations
    df['Clean_City'] = df['City'].str.replace(r' (city|CDP|town|village)$', '',
    regex=True)

    # Split the 'NAME' column ("Abbeville city, Alabama") into two new columns
    df_pop[['Raw_City', 'State_Full']] = df_pop['NAME'].str.split(', ',
    expand=True)

    # again, we need to clean the city name:
    df_pop['Clean_City'] = df_pop['Raw_City'].str.replace(r'
(city|CDP|town|village)$', '', regex=True)

    # Now perform the merge, we have identical combination at both dataframe
    df = pd.merge(df, df_pop, on=['State_Full', 'Clean_City'], how='left')

    # Column Arrangements: (many columns we dont need)
    df = df.rename(columns={'B01003_001E': 'Population'})
    df = df.drop(columns=['Clean_City', 'NAME', 'state', 'place', 'Raw_City' ])
    return df
```

Applying the Function to the DataFrames:

Now, since we defined an optimized function, we only need to apply the function to the DataFrames to add population numbers to every city in the DataFrame.

```
df_poverty = add_population(df_poverty)
df_high_school = add_population(df_high_school)
df_race_shares = add_population(df_race_shares)
df_income = add_population(df_income)
```

Exploring Population-Enhanced Socioeconomic Data

To validate the newly integrated population data and examine the structure of the expanded dataset, we are going to sort the "Income" DataFrame was sorted in descending order to display cities with highest median incomes.

Subsequently, we are going to perform a small filtering to exclude extremely small or unincorporated municipalities, allowing the analysis to focus on real geographic areas.

```
df_income = df_income.sort_values("Median Income", ascending=False)
# Filtering small and unincorporated communities out:
wealthiest_cities = df_income.query("Population > 1000")
print(wealthiest_cities.head(10))
```

	State	State_Full	City	Median Income	Population
17859	NY	New York	Scarsdale village	242,782.00	17,621.00
2537	CA	California	Hidden Hills city	241,667.00	1,557.00
3553	CO	Colorado	Cherry Hills Village city	237,569.00	6,329.00
24837	TX	Texas	Bunker Hill Village city	236,250.00	3,842.00
27085	VA	Virginia	Great Falls CDP	234,091.00	15,764.00
17592	NY	New York	Muttontown village	230,179.00	3,626.00
9568	KS	Kansas	Mission Hills city	229,625.00	3,572.00
2543	CA	California	Hillsborough town	229,097.00	11,253.00
2742	CA	California	Los Altos Hills town	226,319.00	8,314.00
11218	MD	Maryland	Highland CDP	225,337.00	1,113.00

The resulting findings show that the population figures for individual cities have now been successfully incorporated, which will enable more reliable and realistic calculations at the state level in later phases of the analysis.

The results also illustrate why population-weighted averages are essential: Many of the wealthiest places listed above have relatively small populations, meaning that the use of simple arithmetic averages would result in small, wealthy communities disproportionately skewing overall socioeconomic indicators at the state level.

Exporting and Saving the Processed Datasets

After the datasets were enriched with external population data via an API integration, we are going to export the transformed DataFrames and save them as new CSV files to ensure long-term reusability and reproducibility.

In data science workflows, storing processed datasets is a key practice because it reduces unnecessary API requests, improves computational efficiency, preserves preprocessing steps, and ensures that future analyses can be consistently

reproduced without having to repeat time-consuming data collection and transformation processes

Here we define a dictionary with the names and labels of the corresponding datasets to execute the saving and recalling processes efficiently.

```
# Defining dataframes
dataframes = {
    "df_poverty" : df_poverty,
    "df_high_school": df_high_school,
    "df_race_shares" : df_race_shares,
    "df_income": df_income,
}

# Writing and Retriveing the dataframes:
for name, df in dataframes.items():
    df.to_csv(f"csv_files/new_files/{name}.csv", index=False)
    pd.read_csv(f"csv_files/new_files/{name}.csv", encoding="windows-1252")
```

Correcting State-Level Population Bias in Aggregated Data

During the aggregation process in the analysis of the geodata, the sum of the population figures at the city level does not always exactly match the state's total population. This is because the Census API endpoint used for city-level data *state_place* includes only incorporated places and Census Designated Places (CDPs), meaning that residents living in unincorporated rural or suburban areas cannot be in these calculations.

Consequently, the weighted socioeconomic calculations are based exclusively on incorporated cities and designated Places for which detailed data is available.

However, although residents living in unincorporated areas are not included in calculations such as weighted median income or poverty rates, this limitation is not expected to lead to significant distortions, as the datasets for incorporated cities in most states already cover the majority of the population.

To improve the accuracy of population-based comparisons, we also are going to incorporate official total population figures at the state level in the final phase of the calculations to ensure that metrics such as the number of deaths per capita reflect the actual population size of each state.

This approach preserves the reliability of the weighted socioeconomic calculations while allowing for a more realistic normalization of population figures.

Creating a State-Level Population Reference Dataset

To address the issue of under-counting the population, we recall a separate API query to the Census Bureau, using the `state()` endpoint instead of the `state_place()` endpoint. This allows the total population figures for entire states to be retrieved directly from the 2015 ACS dataset.

The returned JSON data is then converted into a structured DataFrame, cleaned by renaming columns and reducing features, and finally exported as a reusable dataset to be integrated into the analysis workflow later.

```
# Pulling Total Population (B01003_001E) for STATES instead of places
state_population_data = c.acs5.state(('NAME', 'B01003_001E'), Census.ALL,
year=2015)

# Converting JSON to a Dataframe:
df_state_pop = pd.DataFrame(state_population_data)

# Renaming and Dropping same columns
df_state_pop.drop(columns=['state'], inplace=True)
df_state_pop = df_state_pop.rename(columns={'B01003_001E':
'total_state_population', 'NAME': 'State_Full'})

# Saving and exporting the datafile:
df_state_population =
df_state_pop.to_csv(f"csv_files/fatal_force_files/new_files/df_state_population.csv", index=False)
```

Designing a State-Level Population Adjustment Function:

We are going to create an additional function to append the officially reported total population figures for the states to the aggregated datasets at performing the final weighted calculations.

By merging DataFrames based on the full names of the states, the function ensures that subsequent statistical analyses are based on more realistic population figures. This helps reduce biases caused by unincorporated areas and improves the overall robustness of state-level socioeconomic indicators.

```
def fix_population(df):  
    # You can now merge this directly with a dataframe of state abbreviations  
    df.drop(columns=['total_state_population'], inplace=True)  
    df = df.merge(df_state_pop, how='left', on='State_Full')  
    col = df.pop("total_state_population")  
    df.insert(2, "total_state_population", col)  
    return df
```

We are going to apply this function to DataFrames after we executed important weighted-property operations.

Now that we have completed the data collection, validation, transformation, and enrichment phases, the datasets are fully prepared for analytical exploration. Through techniques such as data type standardization, feature engineering, API-based data integration, weighted data preparation, and structural optimization of the datasets, a reliable and analysis-ready data environment has been created.

In the following sections, we are going to examine the use of deadly force by police in the United States in 2015 from various socioeconomic and demographic perspectives, including poverty, education, income, ethnicity, geography, age, and gender.

Analysis of Poverty Across the United States

Poverty is one of the most important socioeconomic indicators when examining overall social conditions and regional disparities within the United States. In this section, we are going to analyze how poverty levels vary across states and examine whether regions with higher poverty rates also exhibit different patterns of fatal encounters with the police.

By converting city-level census data into population-weighted state statistics, this analysis aims to provide a more realistic picture of the economic hardship across the country and establish an important socioeconomic baseline for future comparisons with mortality data.

Forming the DataFrame:

To create a reliable dataset on poverty at the state level, the analysis first estimated the exact number of people living in poverty in each city by multiplying the poverty rate of each city by its population.

Using the Pandas *groupby()* function and various aggregation methods, these city-level values are then aggregated into statewide totals, enabling the calculation of population-weighted poverty rates rather than relying on misleading simple averages across cities of vastly different sizes.

Finally, the resulting DataFrame is refined through sorting, rounding, column optimization, and methods for correcting state-level population figures to create a clean and analytically robust dataset suitable for advanced visualizations and comparative analyses.

```
# Adding exact amount of people in poverty at every city:
df_poverty['people_in_poverty'] =
    (df_poverty['poverty_rate'] / 100) * df_poverty['Population']

# State-wise poverty data:
df_poverty_by_state = df_poverty
    .groupby(['State', 'State_Full'], as_index=False)
    .agg(total_state_population=('Population', 'sum'),
        total_state_poverty=('people_in_poverty', 'sum'))

# calculating of poverty rate for each state:
df_poverty_by_state['Poverty_Rate'] = (df_poverty_by_state['total_state_poverty']
    / df_poverty_by_state['total_state_population']) * 100

# Round the number
df_poverty_by_state['Poverty_Rate'] = df_poverty_by_state['Poverty_Rate'].round(2)

# Sort by poverty rate:
df_poverty_by_state = df_poverty_by_state.sort_values('Poverty_Rate',
    ascending=False)

# Fixing population:
df_poverty_by_state = fix_population(df_poverty_by_state)

# Dropping the value for total number of people on poverty would be good, since we
performed an improvement on exact population:
df_poverty_by_state = df_poverty_by_state.drop(columns=['total_state_poverty'])
```

Creating Choropleth Map for the Visualizing Poverty Distribution:

To better understand the geographic distribution of poverty in the United States, an interactive choropleth map is created using Plotly.

The visualization combines a custom color scale, state-level data, and interactive hover information to present poverty rates and population statistics in a clear and visually intuitive map format, making regional differences much easier to identify and interpret.

code:

```
poverty_colorscale = [
    [0.00, "#F6F4D2"], [0.10, "#DCE8B2"], [0.22, "#A8D5A2"], [0.30, "#6DBFA6"],
    [0.40, "#4FA3A5"], [0.50, "#417D9C"], [0.85, "#3F3C7A"], [1.00, "#24152E"]
]

fig = px.choropleth(
    df_poverty_by_state,
    locations='State',
    locationmode='USA-states',
    color='Poverty_Rate',
    scope="usa",
    color_continuous_scale=poverty_colorscale,
    custom_data=['State_Full', 'total_state_population', 'Poverty_Rate']
)

fig.update_traces(
    hovertemplate=(
        "<b>{%customdata[0]} ({%location})</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>Poverty Rate:</b> {%customdata[2]:.2f}%<br>"
        "<b>Total Population:</b> {%customdata[1]:.0f}<br>"
        "<extra></extra>"
    ),
    marker_line_width=0.5, # Thin borders for the states
    marker_line_color='white' # subtle white borders
)

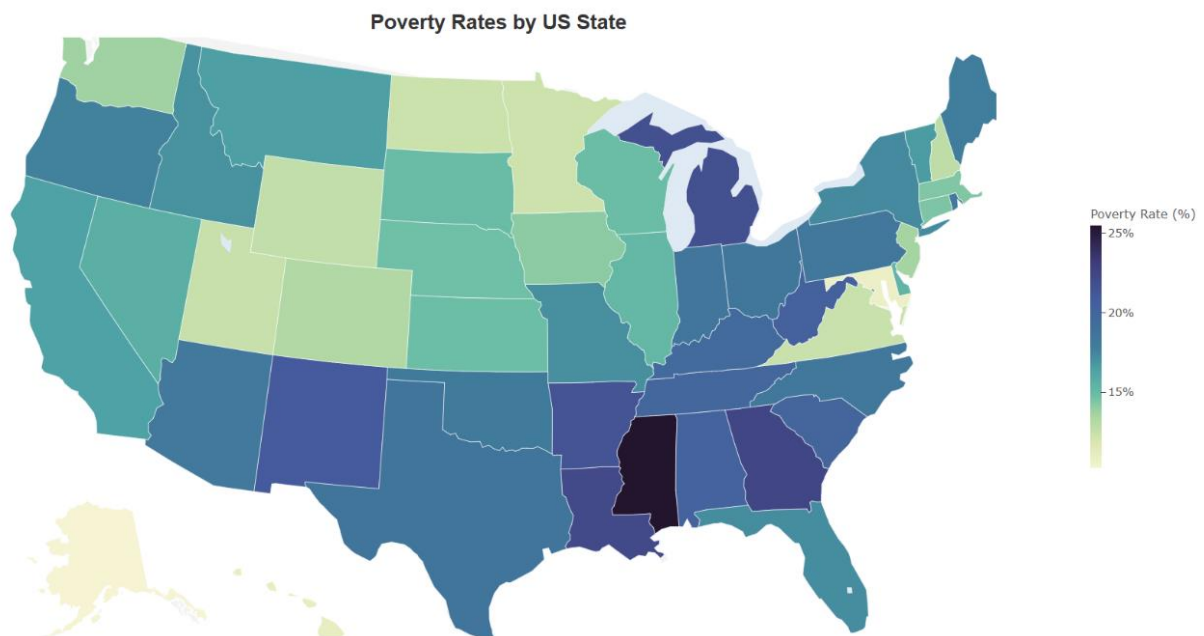
fig.update_layout(
    title=dict(
        text='<b>Poverty Rates by US State</b>',
        font=dict(family="Arial, Helvetica, sans-serif", size=22,
```

```

color="#333333"), x=0.5, y=0.95),
    geo=dict(
        bgcolor='rgba(0,0,0,0)', lakecolor='#E0EAF5', landcolor='#F4F4F4',
        showlakes=True, showcoastlines=False, projection_type='albers usa'
    ),
    coloraxis_colorbar=dict(
        title=dict(
            text="Poverty Rate (%)", font=dict(size=13, color="#555555")),
        thicknessmode="pixels", thickness=12,
        lenmode="pixels", len=300,
        yanchor="middle", y=0.5,
        ticks="outside", ticksuffix="%",
        tickfont=dict(color="#555555")
    ),
    paper_bgcolor='white',
    margin=dict(l=0, r=0, t=60, b=10)
)

fig.show()

```



Interactive Map: [Poverty Rates by US State](#)

The findings show a striking concentration of higher poverty rates in parts of the southern US, with states such as Mississippi, Georgia, Arkansas, and Alabama ranking among the regions with the highest poverty rates in the country in 2015.

Mississippi stands out particularly with a rate of over 25%, which is remarkable considering that the national poverty rate in the United States was significantly lower during this period; this reflects longstanding structural economic challenges in parts of the Deep South, including historical inequality, and restricted economic mobility.

In contrast, states such as Maryland, Alaska, Hawaii, Minnesota, and Virginia have significantly lower poverty rates, suggesting that a more diversified economy and stronger urban economic centers can contribute to lower statewide poverty rates.

Another interesting observation is that several populous states, such as Texas, Florida, California, and New York, still have relatively high poverty rates despite their enormous economic power, illustrating an important socioeconomic reality: high economic performance at the state level does not necessarily lead to an evenly distributed prosperity among the population.

Analysis of Educational Accomplishment

Educational attainment is one of the most meaningful socioeconomic indicators when it comes to examining long-term economic stability, employment opportunities, income levels, and general social trends across different regions. In this section, we are going to investigate the percentage of adults aged 25 and older who have earned a high school diploma in each U.S. state to examine how educational attainment varies geographically across the country.

It is important to note that this metric does not simply represent an annual graduation rate but rather provides a valuable long-term indicator of access to education and the educational attainment of the adult population and helps lay an important foundation for future comparisons with poverty rates, economic conditions, and patterns of fatal police interventions.

Forming the DataFrame:

To create a reliable dataset on high-school diploma rates at the state level, the percentage of adults in each city who had graduated from high school is first

converted into estimated population figures by multiplying the graduation rate by the city's population.

Using grouping and aggregation operations, these city-level values are then aggregated into statewide totals, enabling the calculation of population-weighted graduation rates rather than using statistically misleading simple averages across cities with vastly different populations.

The resulting dataset is subsequently refined through sorting, rounding, population adjustments, and structural cleaning to create a clean, analysis-ready DataFrame for geographic visualization and socioeconomic comparison.

```
df_graduation_by_state = df_high_school.copy()

# Adding exact amount of people in poverty at every city:
df_graduation_by_state['people_completed_hs'] =
(df_graduation_by_state['percent_completed_hs'] / 100) *
df_graduation_by_state['Population']

# State-wise poverty data:
df_graduation_by_state = df_graduation_by_state.groupby(['State', 'State_Full'],
as_index=False).agg(
    total_state_population=('Population', 'sum'),
    total_state_graduates=('people_completed_hs', 'sum')
)

# calculating of poverty rate for each state:
df_graduation_by_state['HS_Graduation_Rate'] =
(df_graduation_by_state['total_state_graduates'] /
df_graduation_by_state['total_state_population']) * 100

# Round the number
df_graduation_by_state['HS_Graduation_Rate'] =
df_graduation_by_state['HS_Graduation_Rate'].round(2)

# Sorting by poverty rate:
df_graduation_by_state = df_graduation_by_state.sort_values('HS_Graduation_Rate',
ascending=False)

# Refining population:
df_graduation_by_state = fix_population(df_graduation_by_state)

df_graduation_by_state =
df_graduation_by_state.drop(columns=['total_state_graduates'])
```

Creating Choropleth Map for the Visualizing Educational Attainment

To better highlight geographical differences in educational attainment across the United States, an interactive choropleth visualization is created using Plotly.

Thanks to custom hover information, a breakdown by state, and a continuous color scale, HS-Graduation rates patterns can be displayed in a highly intuitive way, making regional disparities and broader national trends easier to identify and interpret.

code:

```
graduation_colorscale = [
    [0.00, '#9E5344'], [0.11, '#B25D4D'], [0.22, '#CE8456'], [0.33, '#E2AB67'],
    [0.44, '#EDD488'], [0.55, '#DCE0A6'], [0.66, '#A9C68E'], [0.77, '#71AC7A'],
    [0.88, '#3A8F66'], [1.00, '#0F6B4C']]

fig = px.choropleth(
    df_graduation_by_state,
    locations='State',
    locationmode='USA-states',
    color='HS_Graduation_Rate',
    scope="usa",
    color_continuous_scale= graduation_colorscale,
    custom_data=['State_Full', 'total_state_population', 'HS_Graduation_Rate']
)

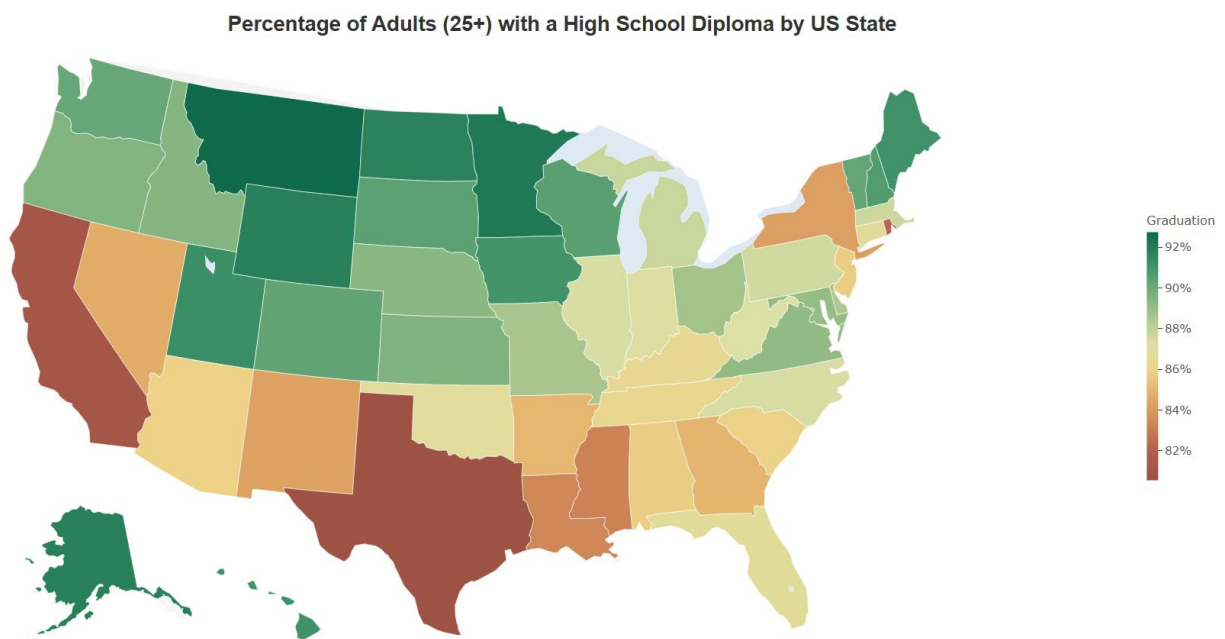
fig.update_traces(
    hovertemplate=(
        "<b>{%customdata[0]} ({location})</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>High-School Graduation Rate:</b> {%customdata[2]:.2f}%<br>"
        "<b>Total Population:</b> {%customdata[1]:,.0f}<br>"
        "<extra></extra>"
    ),
    marker_line_width=0.5, # Thinner state borders
    marker_line_color='white', # Crisp white borders separate
    hoverlabel=dict(bgcolor="#406093", font_family="Arial", bordercolor="#F7F5EB")
)

fig.update_layout(
    title=dict(
        text='<b>Percentage of Adults (25+) with a High School Diploma by US State</b>',
        font=dict(family="Helvetica, sans-serif", size=22, color="#333333"),
        x=0.5, y=0.95),
    geo=dict(
        bgcolor='rgba(0,0,0,0)',
```

```

lakecolor='#E0EAF5', landcolor='#F4F4F4',
showlakes=True,
showcoastlines=False,      # Removes the heavy outline
projection_type='albers usa'
),
coloraxis_colorbar=dict(
    title=dict(text="Graduation Rate",font=dict(size=13, color="#555555")),
    thicknessmode="pixels", thickness=12,
    lenmode="pixels", len=300,
    yanchor="middle", y=0.5,
    ticks="outside", ticksuffix="%",
    tickfont=dict(color="#555555")
),
paper_bgcolor='white',
margin=dict(l=0, r=0, t=60, b=10)
)
fig.show()

```



Interactive Map Link: [Percentage of Adults with a High School Diploma](#)

The choropleth map reveals a clear geographic divide in educational attainment across the United States: Many states in the North, the Midwest, and the Mountain region have the highest percentage of adults with a high school diploma, while several states in the South and the “Sun Belt” have significantly lower rates.

States such as Montana, Minnesota, Wyoming, and North Dakota have a high school graduation rate of over 91%, while Texas and California, despite being the country’s two largest economies, rank near the bottom; this likely reflects the fact that the metric covers the entire adult population aged 25 and older, meaning that factors such as large immigrant populations, language barriers, urban inequality, and highly heterogeneous labor markets can significantly influence educational attainment.

Another interesting aspect of the graph is the relatively small variation between states: even the states with the lowest rates are still above 80%, suggesting that high school graduation is generally widespread among adults in the United States.

However, the concentration of lower educational attainment in the South could still point to broader regional differences in access to education, historical development patterns, migration dynamics, and long-term socioeconomic conditions.

Analysis of the Relationship Between Poverty and Educational Attainment

Poverty and educational attainment are closely linked socioeconomic indicators that often influence long-term economic opportunities, employment stability, and overall quality of life in communities. In this section, we are going to explore whether states with a lower proportion of adults who have completed high school also tend to have higher poverty rates, in order to explore broader structural links between education and the local economic problems in the United States.

By combining these two datasets within a single analytical framework, we aim to identify potential correlations, regional clustering patterns, and broader socioeconomic trends that could explain the inequalities observed across the country.

Forming the DataFrame:

To directly compare educational attainment and poverty rates, the two previously calculated DataFrames are merged into a unified analytical dataset using a multi-column merge function based on shared state identifiers and population values.

This integration process makes it possible to bundle multiple socioeconomic indicators into a single structured DataFrame, enabling correlation analyses, multivariate visualizations, and a more in-depth comparative examination of education and poverty metrics.

```
df_poverty_graduation = df_poverty_by_state
                        .merge(df_graduation_by_state,
                              how='left',
                              on=['State', 'State_Full', 'total_state_population'])
```

Creating Scatter Plot with Trendline for Visualizing the Correlation:

To examine the relationship between poverty and education, an interactive scatter plot is created using Plotly, with each point representing a U.S. state.

In addition to the graphical representation, a regression trendline based on the ordinary least squares (OLS) method is added to the chart, making the general direction and strength of the relationship between the two variables more clearly visible.

code:

```
# Creating the Plot:
fig = px.scatter(
    df_poverty_graduation,
    x='HS_Graduation_Rate',
    y='Poverty_Rate',
    hover_name='State_Full',
    custom_data=['total_state_population'],
    trendline='ols',          # Automatically plots the line of best fit
    trendline_color_override='crimson',
)

# Professional Styling and Hover Templates
fig.update_traces(
    marker=dict(
        size=17,
        color='#0A7C6E',
        opacity=0.6,
        line=dict(width=1, color='rgba(50, 50, 50, 0.8)') # borders around dots
    ),
    hovertemplate=(
        "<b>{%hovertext}</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
    )
)
```

```

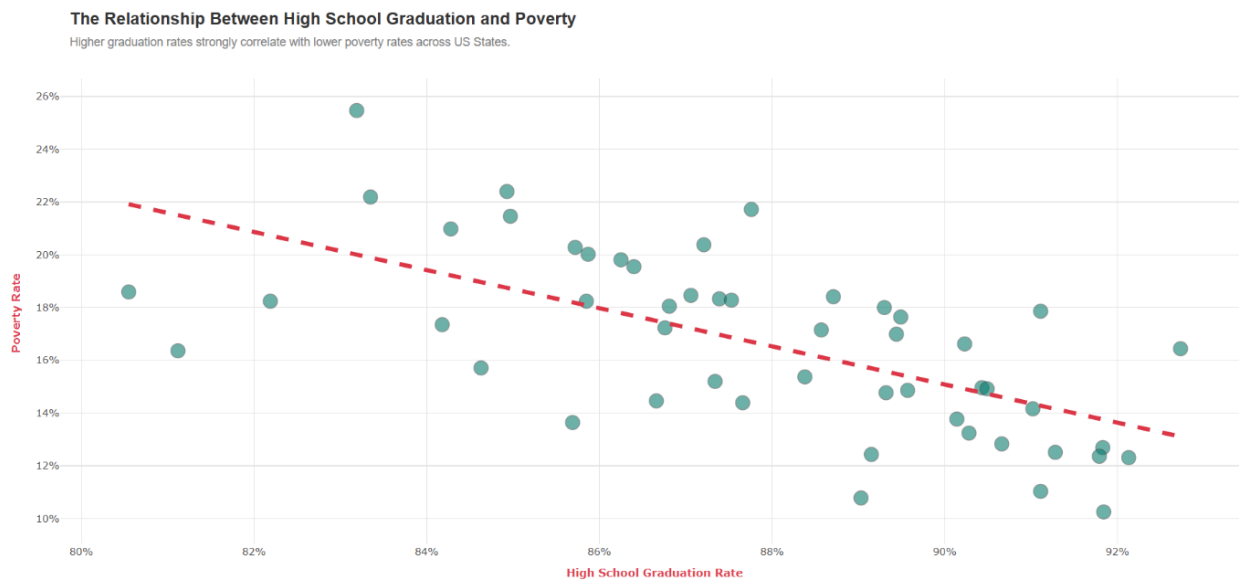
        "<b>Graduation Rate:</b> %{:1f}%<br>"
        "<b>Poverty Rate:</b> %{:1f}%<br>"
        "<b>State Population:</b> %{:0f}"
        "<extra></extra>"
    ),
    selector=dict(mode='markers')
)

trendline_trace = fig.data[1]
# Trendlines
trendline_trace.line.color = '#DC3545'
trendline_trace.line.dash = 'dash'
trendline_trace.line.width = 5

fig.update_layout(
    title=dict(
        text='<b>The Relationship Between High School Graduation and Poverty</b><br>'
        '<span style="font-size: 14px; color: gray;">
            Higher graduation rates strongly correlate with lower poverty rates across
US States.</span>',
        font=dict(family="Arial, sans-serif", size=20, color="#333333"),
        x=0.05, y=0.95
    ),
    xaxis=dict(
        title=dict(
            text="<b>High School Graduation Rate</b>",
            font=dict(size=13, color="#E03F4F")
        ),
        showgrid=True,
        gridcolor='#E5E5E5',
        zeroline=False,
        tickfont=dict(color="#555555"),
        ticksuffix="%"
    ),
    yaxis=dict(
        title=dict( text="<b>Poverty Rate </b>", font=dict(size=13)
        ),
        showgrid=True, gridcolor='#E5E5E5',
        zeroline=False,
        tickfont=dict(color="#555555"), ticksuffix="%"
    ),
    plot_bgcolor='white',
    paper_bgcolor='white',
    margin=dict(l=60, r=40, t=100, b=60),
    hoverlabel=dict( bgcolor="white", font_size=13, font_family="Arial"))
)

fig.show()

```



Interactive graph Link: [Relationship Between High School Graduation and Poverty](#)

The scatter plot shows a clear negative correlation between educational attainment and poverty across U.S. states: as the proportion of adults with a high school diploma increases, poverty rates generally decline.

The downward-sloping regression line reinforces this trend and suggests that educational attainment may play an important role in shaping long-term economic conditions, as higher educational attainment is often associated with better employment opportunities, higher earning potential, and greater economic stability.

At the same time, the graph also highlights some interesting outliers and regional differences—for example, Texas and California have relatively low levels of education but exhibit only moderate poverty rates compared to some Southern states, likely reflecting the influence of large and economically diverse urban economies, immigration patterns, and labor markets that can provide employment opportunities even for populations without formal educational credentials.

Analyzing Median Household Income Across US States

Median household income is one of the most commonly used indicators for assessing the economic well-being, purchasing power, and overall standard of living of a population. In this section, we are going to examine how income levels vary across U.S. states and highlight the broader geographic and socioeconomic patterns that emerge from these differences.

By comparing income distributions across states, this analysis provides an important economic perspective for understanding inequality, regional development, and social conditions, which can later be linked to broader demographic and mortality trends.

Forming the DataFrame:

To create a dataset for a state-level income analysis, median household income values at the city level are first combined with population data to estimate each city's weighted total contribution to income.

Using grouping and aggregation techniques, these weighted values are then aggregated into statewide totals, enabling the calculation of population-adjusted incomes for the states rather than relying on statistically misleading simple averages between cities of different sizes. The resulting dataset is then refined through sorting, rounding, population adjustments, and structural cleaning measures to create a clean, analysis-ready DataFrame for visualization and comparative analysis.

Although official census data provide direct statistics on median income, the state-level income values in this project were deliberately derived from city-level datasets using population-weighted aggregation methods. This approach was chosen to illustrate more advanced data science workflows, including feature engineering, API integration, data set merging and population-weighted statistical calculations.

```
df_income_by_state = df_income.copy()

# Adding exact amount of people in poverty at every city:
df_income_by_state['total_income'] = df_income_by_state['Median Income'] *
df_income_by_state['Population']

# State-wise poverty data:
df_income_by_state = df_income_by_state.groupby(['State', 'State_Full'],
as_index=False).agg(
    total_state_population=('Population', 'sum'),
    total_state_income=('total_income', 'sum')
)
```



```

# calculating of poverty rate for each state:
df_income_by_state['Median_Income'] = df_income_by_state['total_state_income'] /
df_income_by_state['total_state_population']

# Round the number
df_income_by_state['Median_Income'] = df_income_by_state['Median_Income'].round(2)

# Sort by poverty rate:
df_income_by_state = df_income_by_state.sort_values('Median_Income',
ascending=False)

# Fixing population:
df_income_by_state = fix_population(df_income_by_state)

df_income_by_state = df_income_by_state.drop(columns=['total_state_income'])

```

Creating Choropleth Map for the Visualizing Income Distribution

To examine how household income varies geographically across the United States, an interactive choropleth map is created using Plotly. Visual clarity was enhanced through a customized color scale, logarithmic normalization, and interactive hover data, making it easier to identify and interpret differences between states with lower and higher incomes.

code:

```

income_colorscale = [
    [0.00, "#7A1F2B"], [0.10, "#BA444F"], [0.20, "#F3EFEC"], [0.28, "#C6DCCB"],
    [0.60, "#80AE8E"], [0.75, "#579772"], [0.90, "#1F5A3D"], [1.00, "#1A4A32"]
]

fig = px.choropleth(
    df_income_by_state,
    locations='State',
    locationmode='USA-states',
    color=np.log10(df_income_by_state['Median_Income']),
    scope="usa",
    color_continuous_scale=income_colorscale,
    custom_data=['State_Full', 'total_state_population', 'Median_Income']
)

fig.update_traces(
    hovertemplate=(
        "<b>{%customdata[0]} (<{location}</b><br>"

```

```

        "<span style='font-size:10px;'">-----</span><br>"
        "<b>Median Income:</b> ${customdata[2]:,.0f}<br>" # format arrangement
        "<b>Total Population:</b> ${customdata[1]:,.0f}<br>"
        "<extra></extra>"
    ),
    marker_line_width=0.7,
    marker_line_color='white',
    hoverlabel=dict(
        bgcolor="rgba(255,255,255,0.96)",
        bordercolor="rgba(0,0,0,0)",
        font_size=15,
        font_family="Source Sans Pro",
        font_color="#2B2B2B",
        align="left"
    )
)

fig.update_layout(
    title=dict(
        text='<b>Yearly Median Income by US State</b><br>'
        '<span style="font-size: 14px; color: gray;">
            Geographic distribution of well-estimated median household earnings
in 2015.</span>',
        font=dict(family="Arial, Helvetica, sans-serif",
                  size=20, color="#333333"),
        x=0.05, y=0.95
    ),
    geo=dict(
        bgcolor='rgba(0,0,0,0)',
        lakecolor='#FFFFFF',
        landcolor='#F4F4F4',
        showlakes=True,
        showcoastlines=False,
        projection_type='albers usa'
    ),
    coloraxis_showscale=False,
    paper_bgcolor='white',
    margin=dict(l=0, r=0, t=80, b=0) # Tight margins so the map fills the vertical
space
)

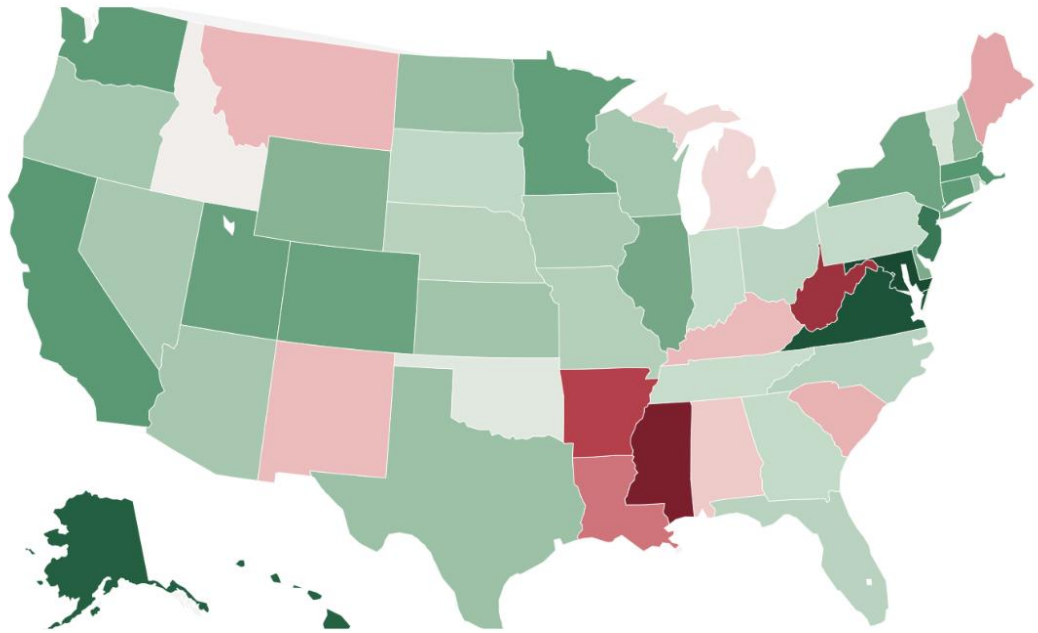
# fig.write_html("interactive_graphs/fatal_force/median_income_by_state.html")

fig.show()

```

Yearly Median Income by US State

Geographic distribution of well-estimated median household earnings in 2015.



Interactive Map Link: [Median Income by US State](#)

The map highlights the significant economic disparities in the United States: Many states in the Northeast, the West, and along the coasts have significantly higher median incomes than large parts of the South and parts of the Appalachian region.

States such as Maryland, Virginia, New Jersey, and Massachusetts are characterized by some of the highest income levels in the country, likely due to their metropolitan areas, high investments in technology and service industries, and proximity to major financial centers such as Washington, D.C., and New York City.

In contrast, states such as Mississippi, West Virginia, Arkansas, and Louisiana remain at the bottom of the distribution, suggesting broader structural economic challenges such as lower educational attainment, less economic concentration in urban areas, and historically higher poverty rates.

Expectedly, several states that previously ranked high in educational attainment analysis, including Minnesota, Colorado, and Washington are also among the higher-income states, underlining that education, labor force specialization, and economic productivity is closely linked at the state level.

Demographic Analysis of Racial Distribution Across US States

The demographic composition varies significantly across different regions of the United States and plays a crucial role in shaping the cultural, economic, political, and social dynamics within states and communities.

In this section, we are going to analyze the ethnic distribution in each U.S. state by examining the population shares of white, Black, Hispanic, Asian, and Native American residents. This allows us to better understand the demographic diversity and geographic concentration patterns that exist across the country.

This analysis provides important contextual background for later sections of the report, particularly when examining inequalities, regional trends, and the demographic composition of data on fatal police interventions.

Forming the DataFrame:

To determine representative ethnic distributions at all states, the demographic shares at the cities are first converted into population figures by multiplying each ethnic share by the corresponding city population—using vectorized operations in Pandas.

These weighted values are then aggregated according to corresponding states and summation operations before being normalized back to percentages. This approach enables the calculation of statistically more accurate, population-weighted demographic distributions.

```
df_race = df_race_shares.copy()

race_cols = ['share_white', 'share_black', 'share_native_american', 'share_asian',
             'share_hispanic']

df_race[race_cols] = df_race[race_cols].multiply(df_race['Population'],
axis="index")

# Grouping by state and sum everything up
df_race_by_state = df_race.groupby(['State', 'State_Full'],
as_index=False)[race_cols + ['Population']].sum()

# Dividing and then converting back to percentages
df_race_by_state[race_cols] =
df_race_by_state[race_cols].divide(df_race_by_state['Population'], axis="index")

df_race_by_state = df_race_by_state.rename(columns={'Population':
'total_state_population'})
```

```
df_race_by_state[race_cols] = df_race_by_state[race_cols].round(2)

df_race_by_state = fix_population(df_race_by_state)

df_race_by_state = df_race_by_state.sort_values('share_white', ascending=False)
```

Correcting Overlapping Demographic Categories:

During preprocessing, a significant issue was identified regarding demographic classification: According to U.S. Census Bureau methods, Hispanic individuals may also be classified as part of the White racial group, which results in the total of demographic shares exceeding 100% in some cases.

To create a clearer and more interpretable visualization, the proportion of the white population was adjusted by excluding overlapping shares of the Hispanic population. This resulted in mutually exclusive demographic categories that better reflect the overall ethnic distribution within each state.

```
# Defining new share_white values:
# By this way, the people in "white" category will only be in that category.
df_race_by_state.share_white = 100.00 - (df_race_by_state.share_black +
df_race_by_state.share_native_american + df_race_by_state.share_asian +
df_race_by_state.share_hispanic)
```

Creating Bar Graph for Racial Demographics by U.S. States

To examine the demographic composition of the United States, a stacked bar chart is created using Plotly, which allows the proportions of different ethnic groups to be visualized simultaneously for each state.

Custom labels, color coding, and interactive custom data information also improve readability and make it easier to compare and interpret the differences in demographic composition between states.

code:

```
race_labels = {
    'share_white': 'White',
    'share_black': 'Black',
    'share_hispanic': 'Hispanic',
```

```

    'share_asian': 'Asian',
    'share_native_american': 'Native American'
}

race_colors = {
    'share_white': '#1A5F7A',
    'share_hispanic': '#F3A712',
    'share_black': '#D9534F',
    'share_asian': '#5CB85C',
    'share_native_american': '#8E6CBE'
}

r_bar = px.bar(
    df_race_by_state,
    x='State_Full',
    y=race_cols,
    barmode='stack',
    color_discrete_map=race_colors,
    custom_data=['total_state_population'],
    labels={"value": "Percentage", "variable": "Demographic", "State_Full": "State"}
)

r_bar.for_each_trace(lambda t: t.update(
    name=race_labels.get(t.name, t.name),
    hovertemplate="<b>{%x}</b><br>{%data.name}: {%y:.1f}%<br><br>"
                  "<span>State Population:</span> {%customdata[0]:.0f}%<br>"
                  "<extra></extra>"
))

# Layout Formatting
r_bar.update_layout(
    plot_bgcolor='white',
    paper_bgcolor='white',
    title=dict(
        text='<b>Racial and Ethnic Composition by State</b><br>'
        '<span style="font-size: 14px; color: gray;">Breakdown of demographic'
        'shares across major US states.</span>',
        font=dict(family="Arial, sans-serif", size=24, color='#2E2E2E'),
        x=0.053, y=0.91
    ),
    xaxis=dict(title="", tickfont=dict(size=12, color='#555555'), showgrid=False),
    yaxis=dict(
        title=dict(text="<b>Percentage</b>", font=dict(size=13, color='#555555'),
                  standoff=20
        ),
        tickfont=dict(size=12, color='#555555'),
        showgrid=True,
        gridcolor='#EAEAEA',
        zeroline=False,
        ticks="outside", # Adding padding to y axis

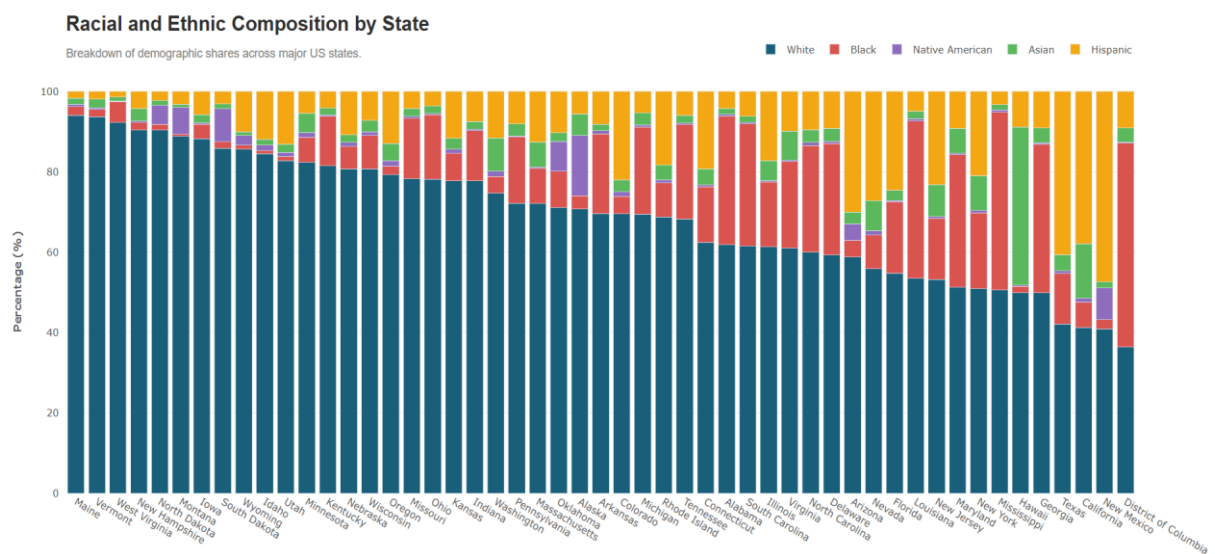
```

```

        ticklen=5,
        tickcolor="white"
    ),
    legend=dict(
        title="",
        orientation="h",
        yanchor="bottom", xanchor="right",
        y=1.02, x=1,
        font=dict(size=12, color='#555555')
    ),
    margin=dict(t=110, l=80, r=40, b=50)
)

r_bar.show()

```



Interactive Graph: [Racial and Ethnic Composition by State](#)

The graph on population distribution illustrates the extraordinary regional diversity of the United States and shows how significantly the racial and ethnic composition varies across different parts of the country.

Northern states such as Maine, Vermont, and West Virginia remain predominantly white, while states in the South and Southwest have far more diverse demographic profiles; for example, Mississippi and the District of Columbia have some of the highest proportions of Black residents, while states such as New Mexico, Texas, and California have an exceptionally high proportion of Hispanic residents, a result of long-term historical migration patterns, proximity to borders, and settlement patterns.

The data also reveals some particularly interesting demographic concentrations: Hawaii has by far the highest proportion of Asian residents in the entire country, Alaska and the Northern Plains states have a higher proportion of Native American residents, and large coastal states such as California, New York, and New Jersey are characterized by a very balanced multiethnic composition, which is likely due to decades of immigration, urbanization, and the economic pull of major metropolitan areas.

Analysis of the Racial Distribution of Fatal Police Encounters

Ethnicity is one of the most discussed and controversial aspects of the debate over police use of force in the United States.

In this section, we are going to explore the ethnic distribution of people killed by police and compare these proportions with the overall ethnic composition of the U.S. population in between 2015 and 2017. In this way, we can examine whether certain population groups are overrepresented in the data on fatalities.

This analysis is not intended to establish direct causal relationships, but rather to identify demographic patterns, highlight potential inequalities, and provide a statistical foundation for a more informed discussion of policing, inequality, and social conditions in the United States.

Forming the DataFrame

To analyze the demographic composition of the death dataset, the records are grouped by ethnic group and then each occurrence counted with aggregation function through unique ID values, which allowed for the efficient calculation of the total number of deaths for each ethnic group.

In addition, estimates of each demographic group's share of the total population are manually entered into the dataset, allowing for a direct comparison between each group's share of total deaths and its approximate share of the total U.S. population.

```
df_fatalities_race_dist = df_fatalities
    .groupby(['race'], as_index=False)
    .agg(Total_Death=('id', 'count'))

# USA race Distribution in 2015-2017:
pop_mapping = {
    'White': 61.5, 'Black': 12.5, 'Hispanic': 17.6,
    'Asian': 5.3, 'Native American': 1.0, 'Other': 2.1
}

df_fatalities_race_dist['US_Pop_Pct'] =
    df_fatalities_race_dist['race'].map(pop_mapping)
```

Creating Pie Chart for the Fatality Demographics

To visually represent the ethnic composition of fatal police encounters, an interactive pie chart is created using Plotly. Custom color coding and embedded hover information on population shares make it easier to compare and interpret the percentage differences between demographic groups in the context of the total population.

code:

```
custom_colors = {
    'White': '#2A3F54',
    'Black': '#1ABB9C',
    'Hispanic': '#E74C3C',
    'Asian': '#F39C12',
    'Native American': '#9B59B6',
    'Other': '#95A5A6'
}

fig = px.pie(
    df_fatalities_race_dist,
    values='Total_Death',
    names='race',
    hole=0.4,
    color='race',
```

```

        color_discrete_map=custom_colors,
        custom_data=['US_Pop_Pct']
    )

# Hover template
fig.update_traces(
    textposition='outside',
    textinfo='percent+label',
    textfont=dict(size=13, color='#555555'),
    marker=dict(line=dict(color='white', width=2)),
    pull=[0.02, 0.02, 0.02, 0.02, 0.02, 0.02],
    hovertemplate=(
        "<b>{%label}</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>Fatalities:</b> {%value:,.0f}<br>"
        "<b>Share of Fatalities:</b> {%percent:.1%}<br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>US Population Share: </b> {%customdata[0][0]}<br>"
        "<extra></extra>"
    )
)

# Layout Formatting
fig.update_layout(
    plot_bgcolor='white',
    paper_bgcolor='white',
    title=dict(
        text="<b>Proportion of Individuals Killed by Police by Race</b><br>"
        "<span style='font-size: 13px; color: gray;'>"
        "Comparing total fatalities alongside 2015 baseline US"
        "demographics.</span>",
        font=dict(size=18, color='#333333'),
        x=0.05, y=0.95
    ),
    showlegend=False,
    margin=dict(t=100, b=40, l=40, r=40)
)

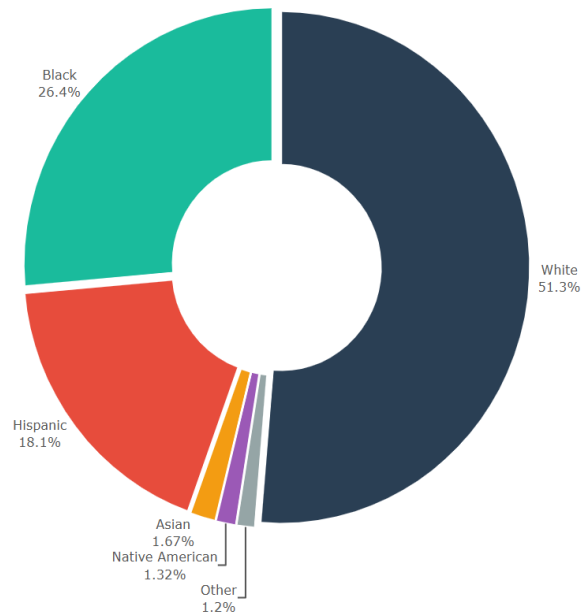
# fig.write_html("interactive_graphs/fatal_force/fatalities_by_race.html")

fig.show()

```

Proportion of Individuals Killed by Police by Race

Comparing total fatalities alongside 2015 baseline US demographics.



Interactive Graph Link: [Proportion of Individuals Killed by Police by Race](#)

The Figure illustrates some notable differences between the ethnic composition of the U.S. population and the ethnic distribution of individuals who died as a result of police interventions in the United States between 2015 and 2017.

Although white individuals account for the largest absolute number of deaths, their share of total deaths (51.3%) is below their approximate share of the total U.S. population in 2015 (61.5%), while Black individuals appear to be significantly overrepresented, accounting for 26.4% of deaths even though they make up only about 12.5% of the total population.

The Hispanic population appears to be involved in fatal encounters with the police more or less proportional to its size, while Native Americans—although they account for only a relatively small proportion of the total number of cases—are also involved in such incidents slightly more often than their share of the total population. This finding is consistent with broader research suggesting that Native American communities experience disproportionate police contact and socioeconomic disadvantages relative to their population size.

Analysis of the Gender Distribution of Fatalities by Police

Gender is another important demographic factor in the study of patterns of fatal police interventions in the United States. In this section, we are going to compare the number of men and women killed by police and examine how these proportions relate to the overall gender composition of the U.S. population in 2015 - 2017.

This analysis provides additional demographic context to help understand which groups are most frequently affected by fatal police shootings, as well as a more comprehensive examination of social behavior, crime risk, police practices, and the distribution of risk across genders.

Forming the DataFrame:

To analyze the gender composition of deaths, the dataset is grouped by gender with aggregating total occurrences, which allows for the efficient calculation of the total number of deaths among men and women from the fatality dataset.

Subsequently, the proportions of the total population for each gender were appended to the DataFrame using mapping operations, allowing for a direct comparison between the representation of each gender in the death data and its approximate proportion of the total U.S. population.

```
df_fatality_sex = df_fatalities
                    .groupby('gender', as_index=False)
                    .agg(Total_Death=('id', 'count'))

# USA race Distribution in 2015:
gender_mapping = {'Female': 51.1, 'Male': 48.9}

df_fatality_sex['US_Gender_Pct'] = df_fatality_sex['gender'].map(gender_mapping)
```

Creating Donut Chart for the Gender Distribution

To illustrate the gender distribution in fatal incidents involving the police in a clear and visually intuitive way, an interactive doughnut chart is created using Plotly.

A custom design, proportional labels, and embedded comparative data on the U.S. population improves readability and highlights the difference between the proportion of men and women in the dataset on fatal incidents

code:

```
gender_colors = {'Female': '#D14D72', 'Male': '#355C7D'}

fig = px.pie(
    df_fatality_sex,
    values='Total_Death',
    names='gender',
    hole=0.4,
    color='gender',
    color_discrete_map=gender_colors,
    custom_data=['US_Gender_Pct']
)

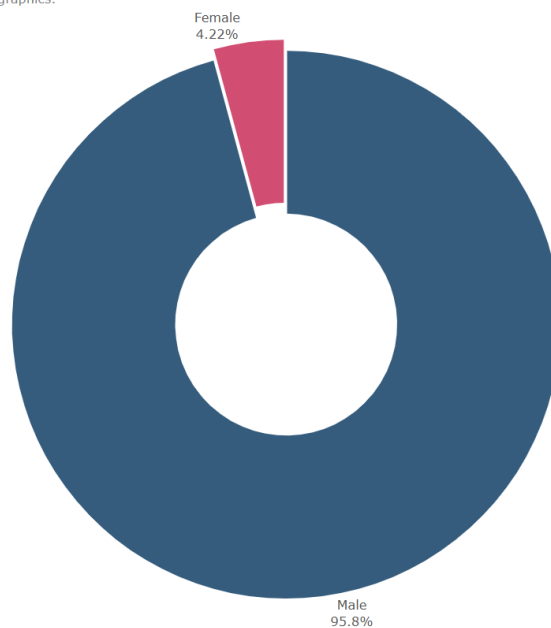
# Hover template
fig.update_traces(
    textposition='outside',
    textinfo='percent+label',
    textfont=dict(size=13, color='#555555'),
    marker=dict(line=dict(color='white', width=2)),
    pull=[0.02, 0.02, 0.02, 0.02, 0.02, 0.02],
    hovertemplate=(
        "<b>{%label}</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>Fatalities:</b> {%value:,.0f}<br>"
        "<b>Share of Fatalities:</b> {%percent:.1%}<br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>US Population Share: </b> {%customdata[0][0]}<br>"
        "<extra></extra>"
    )
)

# Layout Formatting
fig.update_layout(
    plot_bgcolor='white',
    paper_bgcolor='white',
    title=dict(
        text="<b>Sex Distribution of Individuals Killed by Police by Race</b><br>"
        "<span style='font-size: 13px; color: gray;'>Comparing total"
        "fatalities alongside 2015 baseline US demographics.</span>",
        font=dict(size=22, color='#333333'),
        x=0.05, y=0.95
    ),
    showlegend=False,
```

```
margin=dict(t=100, b=40, l=40, r=40)
)
fig.show()
```

Sex Distribution of Individuals Killed by Police by Race

Comparing total fatalities alongside 2015 baseline US demographics.



Graph Link: [Sex Distribution of Individuals Killed by Police by Race](#)

The chart shows an extremely skewed gender distribution in fatal incidents involving the police: men account for 95.8% of all recorded deaths, even though they make up less than half of the total U.S. population.

This disparity is likely influenced by several intertwined social and behavioral factors—including the fact that men are statistically far more likely to be involved in violent crimes, armed and high-risk confrontations, and aggressive police interventions—and the findings are consistent with broader criminological research showing that men in the United States constitute the vast majority of homicide victims, perpetrators of violence, prison inmates, and incidents involving police use of force.

Analysis of Fatal Police Encounters by Gender and Race

Now that we have examined the distribution by ethnicity and gender separately within the dataset on deaths, the next step is to analyze how these two demographic dimensions intersect. In this section, we are going to examine the number of men and women killed by police within each ethnic category to determine whether the significant gender imbalance previously observed remains consistent across the various demographic groups.

This combined analysis provides a more detailed demographic perspective on the dataset and helps to illustrate how ethnicity and gender together shape the overall distribution of fatal police interventions in the United States.

Forming the DataFrame:

To conduct a multidimensional demographic analysis, the dataset is first filtered to exclude records with unspecified or ambiguous ethnic affiliations.

The remaining death records are then grouped by race and gender with aggregating the counts of total occurrences, allowing the total number of deaths to be calculated simultaneously by ethnicity and gender before being sorted by total number of deaths in the visualization for greater clarity.

```
# We won't include the entries with no race data:
df_fatalities_filtered = df_fatalities.query("race != 'Other'")

# Counting by gender and by race:
df_fatalities_by_gender_race = df_fatalities_filtered.groupby(['race', 'gender'],
as_index=False).agg(Total_Death=('id', 'count'))

# Sorting by total number of individuals
df_fatalities_by_gender_race =
df_fatalities_by_gender_race.sort_values('Total_Death', ascending=False)
```

Creating Grouped Bar Graph:

To compare the number of deaths among men and women in different ethnic groups, a grouped bar chart is created using Plotly.

By arranging the bars side by side, using a custom color scheme, and including specific numerical values, the chart's readability is improved, making it easier to visually interpret the demographic differences between ethnic groups and genders.

code:

```
fig = px.bar(
    df_fatalities_by_gender_race,
    x='race',
    y='Total_Death',
    color='gender',
    barmode='group', # bars side by side
    text='Total_Death',
    color_discrete_map=gender_colors,
    category_orders={'Race': df_fatalities_by_gender_race} # Sorting x-axis
)

# designing the graph
fig.update_traces(
    textposition='outside', # Pushes the labels above the bars
    textfont=dict(size=12, color='#333333'),
    marker_line_width=0, # no borders for the bars
    hovertemplate=(
        "<b>{x}</b> ({data.name})<br>"
        "<b>Individuals:</b> {y}<br>"
        "<extra></extra>"
    )
)

fig.update_layout(
    plot_bgcolor='white',
    paper_bgcolor='white',
    title=dict(
        text='<b>Fatalities by Race and Gender</b><br>'
        '<span style="font-size: 14px; color: gray;">Male fatalities'
        'drastically outnumber female fatalities across all demographic groups.</span>',
        font=dict(family="Arial, sans-serif", size=20, color='#333333'),
        x=0.05, y=0.93
    ),
    xaxis=dict(title="", tickfont=dict(size=12, color='#555555'), showgrid=False),
    yaxis=dict(
        title=dict(
            text='<b>Number of Individuals</b>',
            font=dict(size=15, color='#555555'), standoff=20
        ),
        tickfont=dict(size=12, color='#555555'),
        showgrid=True, gridcolor='#EAEAEA',
        zeroline=False,
```

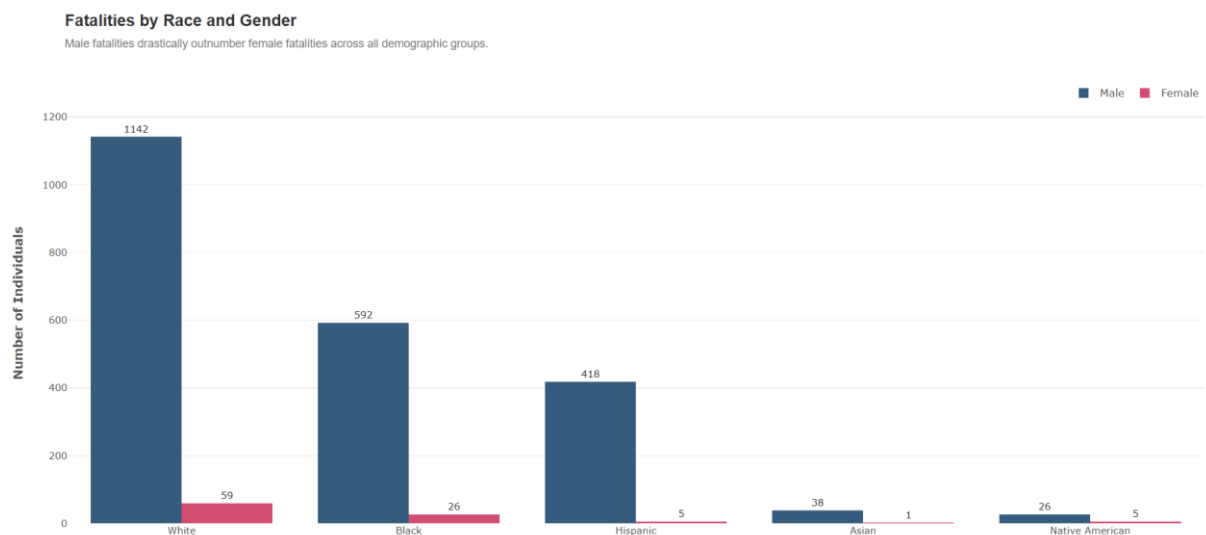


```

        range=[0, 1300]
    ),
    legend=dict(
        title="", orientation="h",
        yanchor="bottom", y=0.95,
        xanchor="right", x=1,
        font=dict(size=13, color='#555555')
    ),
    margin=dict(t=120, l=80, r=40, b=50)
)

fig.show()

```



Graph Link: [Fatalities by Race and Gender](#)

The grouped bar chart shows that the vast majority of deaths in each ethnic group involve men, further underscoring the significant gender imbalance noted in the previous section.

White and Black men account for the largest absolute number of deaths, but what is particularly striking is the extremely low number of female deaths across all groups—most notably, there is only a single Asian woman appears in the entire dataset.

This individual is 37-year-old Sinuon Pream of Long Beach, California, a woman of Cambodian descent with a mental illness who was shot and tasered by police in 2017 while reportedly holding a small knife; the case later garnered widespread national attention after a jury awarded her family millions of dollars and criticized the officers' actions in light of her mental health crisis.

Analysis of the Age Distribution of Individuals Killed by Police

Age is another important demographic variable that can shed light on which population groups in the United States are most frequently affected by deaths related to police encounters.

By analyzing the age distribution of individuals in the dataset, we can better understand whether deaths are concentrated among younger adults, middle-aged adults, or older adults, and assess how factors such as crime risk, socioeconomic status, mental health issues, or patterns of police contact differ across age groups.

This section therefore aims to identify broader demographic trends within the dataset on deaths and to provide additional context for interpreting the social characteristics of the population in question.

Creating the Histogram

To illustrate the age distribution, a histogram graph is created using the Seaborn statistical visualization library, which is particularly well-suited for displaying frequency distributions and identifying patterns of concentration in continuous numerical variables.

The histogram groups individuals into age intervals (bins) and allows us to identify the age groups in which deaths are most concentrated, while stylistic adjustments such as transparency, edge coloring, and high-resolution rendering improve readability and visual clarity for analytical interpretation.

code:

```
sns.set_theme(style="white", context="notebook")
plt.figure(figsize=(12, 6), dpi=300)

ax = sns.histplot(
    data=df_fatalities,
    x='age',
    bins=80,
    color="#3465a4",
    edgecolor="white",
    linewidth=1.5,
    alpha=0.9
)

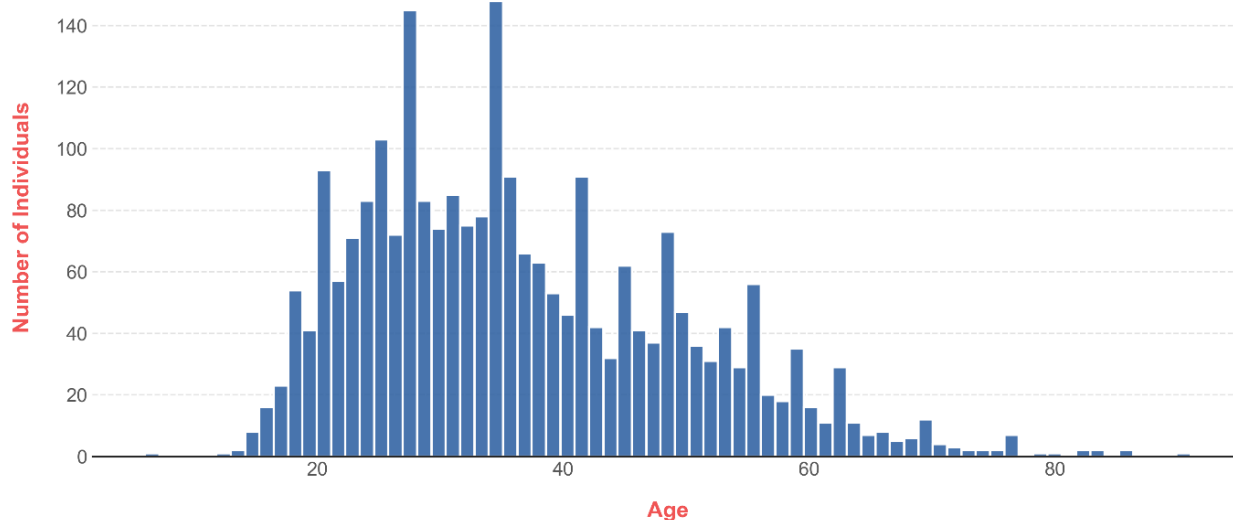
plt.title('Age distribution of the Suspects who killed by police',
          fontsize=22, pad=20, fontweight='bold', color='#F05454')
plt.xlabel('Age', fontsize=15, labelpad=15, fontweight='bold', color='#F05454')
plt.ylabel('Number of Individuals', fontsize=15, fontweight='bold', labelpad=15,
          color='#F05454')

ax.grid(axis='y', color='#dddddd', linestyle='--', alpha=0.8)
ax.tick_params(axis='both', which='major', labelsize=13, colors='#555555',
length=0)
ax.set_axisbelow(True)

sns.despine(left=True, bottom=False)
plt.tight_layout()

# plt.show()
```

Age distribution of the Suspects who killed by police



The histogram shows that deaths related to police interventions between 2015 and 2017 were heavily concentrated among young and middle-aged adults, particularly those in their early 20s to late 30s, with the highest number of deaths occurring in the 25- to 36-year-old age group.

This pattern is not entirely surprising when compared to broader criminological and security policy studies, as younger adult men are statistically more likely to come into contact with the police, be exposed to violent crime, become involved in armed confrontations, participate in gang-related crime, and engage in risky behavior—all factors that can increase the likelihood of fatal incidents with the police.

On the other hand, the number of deaths drops significantly after age 50, suggesting that older age groups are significantly less likely to be involved in high-risk interactions with the police.

Another notable observation is that, although the number is relatively small compared to groups of adults, the dataset nevertheless includes several minors and even very young individuals including documented deaths of teenagers and children, which makes it clear that police violence in the United States is not limited exclusively to the adult population; an issue that is examined in greater detail in the following section.

Examining Child Fatalities Caused by Police

When examining the histogram of age distribution, one of the most disturbing findings is that the dataset includes not only adults but also several children and very young adolescents who lost their lives in confrontations with the police.

To investigate this issue, the dataset is filtered to include only individuals aged 15 or younger, allowing us to examine these extremely concerning cases more closely.

Firstly, the data is narrowed down via a conditional query (`age <= 15`) and then sorted by ascending age to prioritize the youngest victims. At the same time, a subset of relevant descriptive columns is selected to provide a clearer and more interpretable overview of the incidents.

```
# Filtering for under 15 and sorting ascendingly by age:
fatalities_under_15 = df_fatalities.query('age <= 15')

fatalities_under_15 = fatalities_under_15.sort_values("age", ascending=True)

# We need to plot what features we would like to see here:
display_columns = ['name', 'date', 'manner_of_death', 'armed', 'age', 'gender',
                   'race', 'State_Full']
```

Children Killed by Police in the USA:

name	date	manner_of_death	armed	age	gender	race	State_Full
Jeremy Mardis	2015-03-11	shot	unarmed	6	Male	White	Louisiana
Ciara Meyer	2016-11-01	shot	unarmed	12	Female	White	Pennsylvania
Tyre King	2016-09-14	shot	toy weapon	13	Male	Black	Ohio
Jesse James Romero	2016-09-08	shot	gun	14	Male	Hispanic	California
Jorge Santiago Tapia	2015-10-15	shot	gun	15	Male	Hispanic	Florida
Andre Green	2015-09-08	shot	vehicle	15	Male	Black	Indiana
Zane Terryn	2015-06-14	shot	gun	15	Male	White	Florida
Jorevis Scruggs	2016-04-19	shot	gun	15	Male	Black	Missouri
Luke Smith	2016-11-19	shot and Tasered	knife	15	Male	Asian	California
Daezion Turner	2017-04-24	shot	gun	15	Male	NaN	Texas
Jordan Edwards	2017-04-29	shot	unarmed	15	Male	Black	Texas
Jacob Peterson	2017-06-05	shot	toy weapon	15	Male	White	California

The cases listed above are among the most controversial and emotionally devastating incidents of police shootings between 2015 and 2017, as the victims were children or very young teenagers whose deaths sparked nationwide media coverage and protests about the use of force by police and their accountability.

The most shocking case involved six-year-old Jeremy Mardis of Louisiana, who was killed after police officers fired multiple shots at his father's vehicle during a car chase; unlike most police shooting cases, one of the officers involved was later convicted and sentenced to prison.

Another major case involved 15-year-old Jordan Edwards of Texas, an unarmed Black teenager who was shot while sitting in a car that was leaving a party. This sparked nationwide outrage after body camera footage contradicted the police's initial account and led to a rare murder conviction for the officer involved.

Several other cases, including those of Tyre King and Andre Green, also became part of a broader national debate on racism, policing, and how the police handle minors. A key pattern in the data is that nearly all child fatalities involved boys, while many incidents escalated because officers believed the children were in possession of weapons such as air rifles, toy guns, knives, or vehicles.

Overall, these tragedies have intensified the public debate on police reform, de-escalation training, body cameras, and the question of whether law enforcement agencies are sufficiently prepared to handle encounters with children and adolescents without resorting to lethal force.

Analysis of Age Distribution by Race

In this section, we are going to examine how the age distribution of deaths resulting from police encounters in the United States varies by ethnic group.

The combined analysis of age and ethnicity provides a more detailed demographic picture of the dataset and allows us to determine whether certain ethnic groups disproportionately appear in fatal police encounters among younger or older age groups.

Creating the Box Plot for Statistical Analysis

To statistically compare the age distributions of different ethnic groups, a box plot visualization is created using Plotly Express.

Box plots are particularly useful in data science because they summarize the median, interquartile range, overall dispersion, and potential outliers within each demographic category. This allows us to compare not only average trends but also variability and extreme cases across ethnic groups.

code:

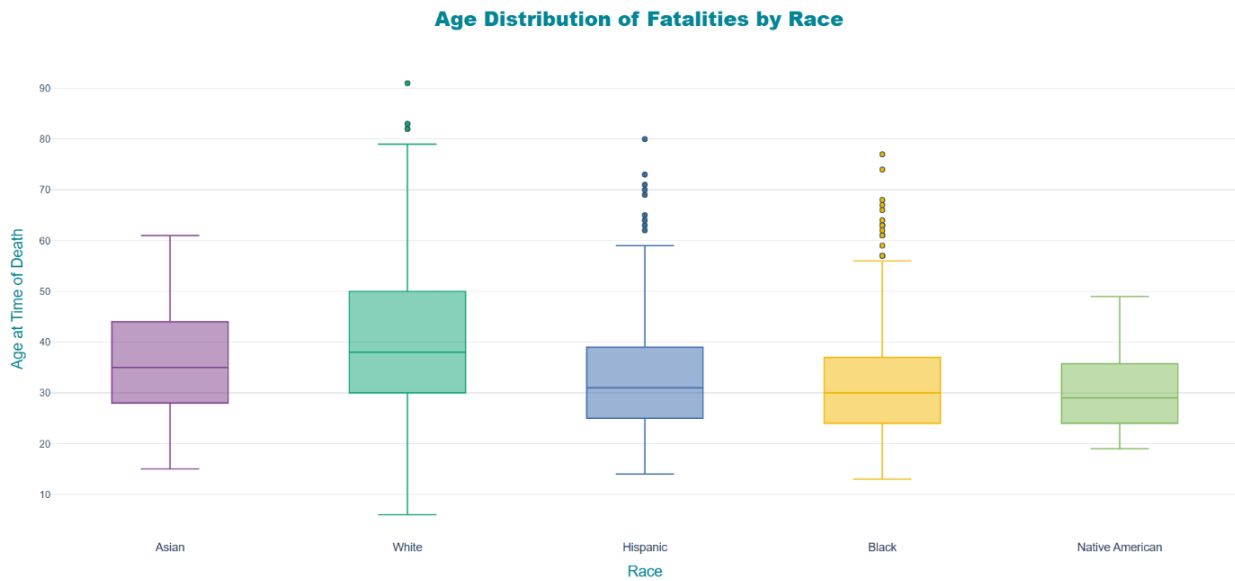
```
fig = px.box(
    df_fatalities_filtered,
    x="race",
    y="age",
    color="race",
    title='Age Distribution of Fatalities by Race',
    labels={"race": "Race", "age": "Age at Time of Death"},
    color_discrete_sequence=px.colors.qualitative.Bold,
    template="plotly_white"
)

fig.update_layout(
    font_family="Inter, Helvetica, Arial, sans-serif",
    title_font_size=24, title_font_color="#008695", title_font_family="Arial",
    title_x=0.5, # Center the title
    showlegend=False, # No need to show Legend
    xaxis=dict(showgrid=False, zeroline=False, title_font_size=18,
               tickfont=dict(size=14), title_font_color="#008695"),
    yaxis=dict(showgrid=True, gridcolor='#E5E7EB', zeroline=False,
               title_font_size=18, title_font_color="#008695"),
    plot_bgcolor='rgba(0,0,0,0)',
    margin=dict(t=80, b=40, l=40, r=40)
)

# Polish the boxes themselves (line width and outlier styling)
fig.update_traces(
    marker=dict(size=6, opacity=1, line=dict(width=1, color='DarkSlateGrey')),
    line=dict(width=1.5)
)

# fig.write_html("interactive_graphs/fatal_force/age_distribution_by_race.html")

fig.show()
```



Interactive Graph Link: [Age Distribution of Fatalities by Race](#)

The box plot shows significant differences in the age distribution of deaths related to police encounters among different ethnic groups.

White individuals have the highest median age at the time of death (38 years) as well as the widest age range, encompassing both the youngest fatality recorded in the data (6 years old) and the oldest person (91 years old). This suggests that fatal police encounters with white individuals occur across a broader spectrum of life stages.

In contrast, Black, Hispanic, and Native American victims tend to be concentrated in younger age groups with a median age of approximately 29–31 years, suggesting that younger adults from these groups are disproportionately involved in fatal police-involved incidents.

Overall, younger men from minority groups come into contact with the police more frequently—a trend that can be attributed to factors such as the intensity of policing in urban areas, socioeconomic inequality, concentrated poverty, police strategies related to gang crime, and a higher rate of proactive policing in disadvantaged communities, while the relatively higher average age of Asian and white victims may be partly attributable to differing demographic age structures, patterns of criminal vulnerability, and the dynamics of policing within these populations.

Analysis of Weapon Possession Among Individuals Killed by Police

One of the most debated aspects of police-involved fatalities in the United States is whether the individuals involved were armed at the time of the encounter, as this factor is often central to discussions about justification, the appropriate use of force, officer safety, and public accountability.

In this section, we are going to explore how many of the individuals killed by police were armed and how many were unarmed, while also shedding light on the overall distribution of gun ownership within the dataset.

Understanding these dynamics is important because it provides a deeper context for evaluating police actions and helps distinguish between cases involving an immediate lethal threat and incidents where lethal force may have been used under more controversial circumstances.

Forming the DataFrame:

To perform this analysis, the dataset on deaths is duplicated and expanded to include a new categorical feature for arm possession which divides the individuals into two simplified groups based on the original “armed” column.

By using the vectorized NumPy function `np.where()`, the classification process could be efficiently applied to the entire dataset. The data is then grouped by this new feature and aggregated to calculate the total number of deaths in each category for statistical comparisons and visualizations.

```
df_killed_armed = df_fatalities.copy()

# Filtering the dataframe and creating new through an existing column
df_killed_armed['armed_or_not'] = np.where(
    df_killed_armed.armed == 'unarmed', 'no', 'yes')

# Grouping and counting
df_killed_armed_or_not = df_killed_armed
    .groupby('armed_or_not', as_index=False)
    .agg(Total_Death=('id', 'count'))
```

Creating the Donut Chart

To illustrate the relationship between armed and unarmed individuals, a pie (donut) chart is created using Plotly, which allows for an intuitive comparison.

Donut charts are particularly well-suited for representing part-whole relationships and convey at a glance the proportion of armed individuals relative to unarmed cases within the entire dataset of fatalities.

code:

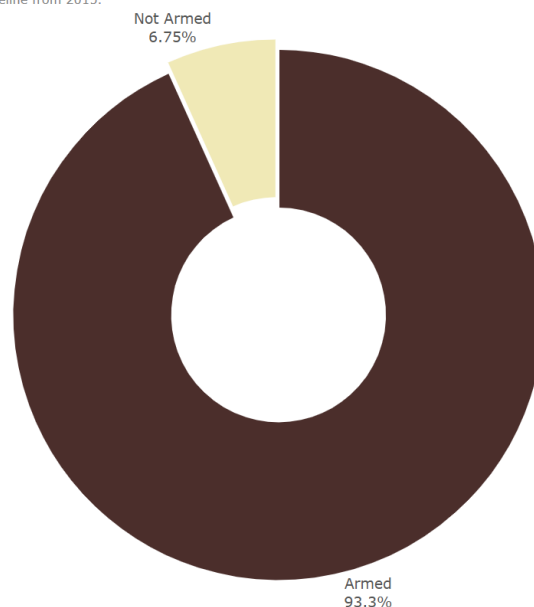
```
fig = px.pie(
    df_killed_armed_or_not,
    values='Total_Death',
    names=['Not Armed', 'Armed'],
    hole=0.4,
    color='armed_or_not',
    color_discrete_map=armed_colors,
)

# Hover template
fig.update_traces(
    textposition='outside', textinfo='percent+label',
    textfont=dict(size=15, color='#555555'),
    marker=dict(line=dict(color='white', width=2)),
    pull=[0.02, 0.02, 0.02, 0.02, 0.02, 0.02],
    hovertemplate=(
        "<b>{%label}</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>Fatalities:</b> {%value:,.0f}<br>"
        "<b>Share of Fatalities:</b> {%percent:.1%}<br>"
        "<extra></extra>"
    )
)

# Layout Formatting
fig.update_layout(
    plot_bgcolor='white',
    paper_bgcolor='white',
    title=dict(
        text="<b>Were the Fatalities Armed ?</b><br>"
        "<span style='font-size: 13px; color: gray;'>Comparison of the total"
        "number of deaths with the U.S. baseline from 2015.</span>",
        font=dict(size=22, color='#333333'),
        x=0.05, y=0.95
    ),
    showlegend=False,
    margin=dict(t=100, b=40, l=40, r=40)
)
fig.show()
```

Were the Fatalities Armed ?

Comparison of the total number of deaths with the U.S. baseline from 2015.



It is not called “Donut Chart” for nothing. 😊

Graph Link: [Were the Fatalities Armed ?](#)

The chart shows that approximately 93.3% of the people killed by police were classified as armed, while about 6.7%—or 171 people—were reported as unarmed; a proportion that may seem small in numerical terms but remains socially and legally significant given the irreversibility of lethal force.

Although U.S. law generally permits police officers to use deadly force when they reasonably perceive an immediate threat to life, many cases involving unarmed individuals—including highly publicized incidents such as those involving Walter Scott, Daniel Shaver, Samuel DuBose, and Justine Damond—sparked nationwide controversy, as video footage or witness accounts raised serious questions regarding necessity, proportionality, and police decision-making under pressure.

Several police officers involved in the killing of unarmed individuals have faced criminal charges, dismissals, or civil lawsuits—for example, former police officer Michael Slager was convicted at the federal level following the killing of Walter Scott, while the officer who killed Justine Damond was convicted of murder and later released—yet many other cases resulted in no convictions due to legal standards such as “objective reasonableness,” protection under qualified immunity, difficulties in proving intent, and the fact that courts often grant police officers broad sensitivity in rapidly evolving situations, did not result in convictions.

Analysis of Weapon Types Possessed During Police Fatalities

In this section, we examine the specific types of weapons or objects that individuals killed by police were reportedly carrying in order to better understand the circumstances of these incidents and identify the types of threats to which police officers were most frequently exposed.

Analyzing weapon categories provides more detailed insights than a simple comparison between armed and unarmed cases. This allows us to assess how often firearms, knives, vehicles, toy guns, or other objects played a role in fatal incidents, and this helps us interpret broader patterns in situations where police use force.

Forming the DataFrame:

To prepare this analysis, we first the dataset is filtered to exclude the unarmed individuals.

Then the dataset is grouped and aggregated to determine the total number of deaths for each weapon category, while the `.assign()` method, combined with a lambda function, dynamically calculates each category's percentage of the total number of shooting deaths; this approach ensures that the transformation remains fully vectorized and efficient, and that the percentages are automatically recalculated as soon as the dataset changes.

```
# Filtering for the armed
df_killed_arm_type = df_killed_armed[df_killed_armed['armed'] != 'unarmed']

# Renaming the column
df_killed_arm_type = df_killed_arm_type.rename(columns={'armed': 'Arm_Type'})

# Grouping by arm type to get their total numbers and percentages:
df_killed_arm_type = (
    df_killed_arm_type
    .groupby('Arm_Type', as_index=False)
    .agg(Total_Individuals=('id', 'count'))
    .assign(
        Arm_Share=lambda x: ['Total_Individuals'] / ['Total_Individuals'].sum() * 100
    )
)

# Only more than 2 (otherwise there are too many options)
df_killed_arm_type = df_killed_arm_type.query("Total_Individuals > 2")

df_killed_arm_type = df_killed_arm_type.sort_values("Total_Individuals",
ascending=False)
```

Creating Horizontal Bar Graph:

To visually illustrate the prevalence of different weapon categories in police-involved fatal incidents, a horizontal Plotly bar chart is used, while a logarithmic color scale helps to visually highlight the significant differences between very common weapon types and far less common categories.

code:

```
arm_bar = px.bar(
    df_killed_arm_type,
    x='Total_Individuals',
    y='Arm_Type',
    orientation='h',
    custom_data=['Arm_Share'],
    text='Total_Individuals',
    color=np.log10(df_killed_arm_type['Total_Individuals']),
    color_continuous_scale='Matter',
)

arm_bar.update_traces(
    # marker_color='#2A3F54',
    marker_line_width=0,
    textposition='outside',
    texttemplate='%{x:,.0f}',
    textfont=dict(size=12, color='#333333'),
    cliponaxis=False,
    hovertemplate=(
        "<b>{%y}</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>Total Individuals:</b> {%x:,.0f}<br>"
        "<b>Share of Total:</b> {%customdata[0]:.1f}%<br>"
        "<extra></extra>"
    )
)

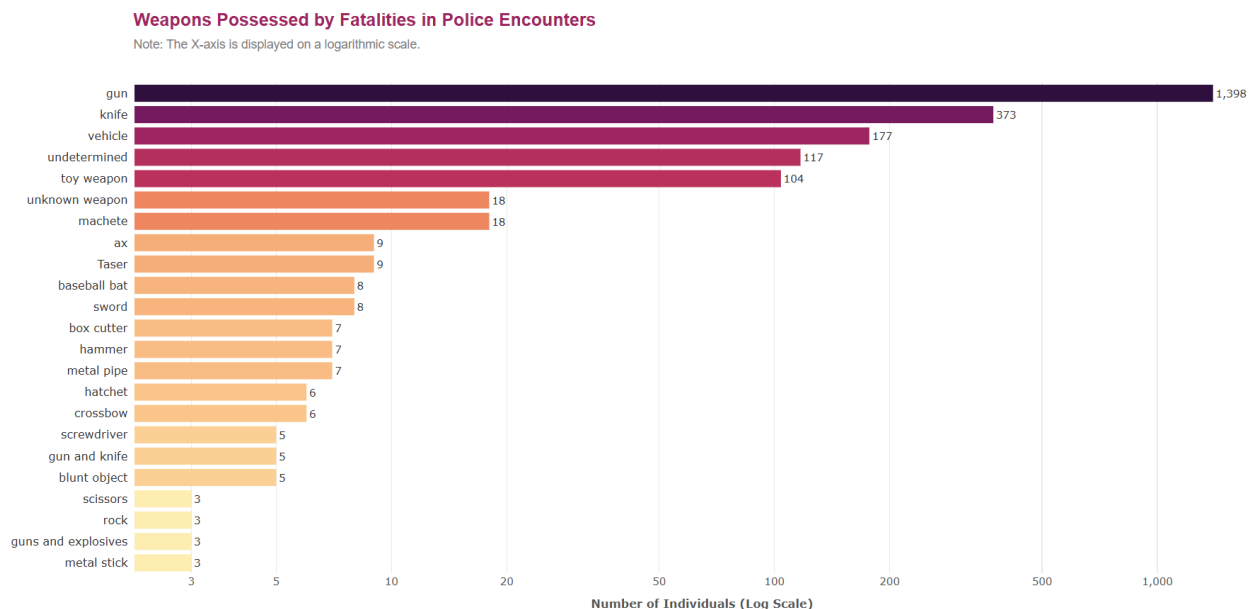
arm_bar.update_layout(
    plot_bgcolor='white', paper_bgcolor='white',
    title=dict(
        text='<b>Weapons Possessed by Fatalities in Police Encounters</b><br>'
        '<span style="font-size: 14px; color: gray;">Note: The X-axis is'
        'displayed on a logarithmic scale.</span>',
        font=dict(family="Arial, sans-serif", size=20, color='#9f2561'),
        x=0.115, y=0.95
    ),
    xaxis=dict(
        title=dict(
            text="<b>Number of Individuals (Log Scale)</b>",
            font=dict(size=13, color='#555555'),
```

```

        standoff=15
    ),
    type='log',
    # customized x-axis labels:
    tickvals=[3, 5, 10, 20, 50, 100, 200, 500, 1000],
    ticktext=['3', '5', '10', '20', '50', '100', '200', '500', '1,000'],
    tickfont=dict(size=12, color='#555555'),
    showgrid=True, gridcolor='#EAEAEA', zeroline=False
),
yaxis=dict(
    title="",
    categoryorder='total ascending',
    tickfont=dict(size=13, color='#333333'),
    showgrid=False, zeroline=False,
    # Y-axis Padding:
    ticks="outside", ticklen=4, tickcolor="white"
),
margin=dict(t=100, l=120, r=60, b=60),
coloraxis_showscale=False,
)

fig.show()

```



Interactive Graph Link: [Weapons Possessed by Fatalities in Police Encounters](#)

The chart shows that guns play an overwhelming role in fatal police encounters, accounting for nearly 60% of all armed incidents, while knives represent the second-largest category at about 16%. This suggests that in most fatal police encounters, weapons are involved that officers perceive as an immediate threat to their lives.

Another notable finding is the relatively high number of fatalities involving vehicles and toy guns, particularly following several incidents that garnered nationwide attention—such as shootings involving toy guns or imitation weapons—many police departments expanded their de-escalation training and body camera programs in the mid-2010s, reflecting growing public concern about how quickly police officers must distinguish between real and perceived threats under stressful conditions.

Analysis of Mental Illness in Police Fatality Cases

In this section, we are going to examine how frequently signs of mental illness were identified in individuals killed by the police which is an issue that is gaining increasing prominence in discussions about crisis intervention and public safety.

Analyzing mental illness helps us better understand whether police officers often respond not only to criminal situations but also to psychiatric or emotional crises that may require specialized de-escalation strategies rather than force-based responses.

This analysis also contributes to broader debates regarding the availability of mental health services, the training of police officers for crisis situations, and the growing role of the police as first responders to mental health emergencies.

Forming the DataFrame:

To prepare for the analysis, the dataset is first grouped based on the Boolean column *signs_of_mental_illness* and aggregated to determine the total number of deaths in each category.

This technique efficiently summarizes categorical data into meaningful proportions, allowing us to directly compare cases whether mental illness was reported.

```
df_killed_mental_illness = df_killed_armed.groupby('signs_of_mental_illness',  
as_index=False).agg(Total_Death=('id', 'count'))
```

```
mentally_ill = {False: '#35858E', True: '#E03F4F'}
```

Creating the Pie Chart

To visually illustrate the percentage distribution between deaths with reported signs of mental illness and those without such signs a pie chart is applied, so that the imbalance and the full extent of the problem are immediately apparent.

code:

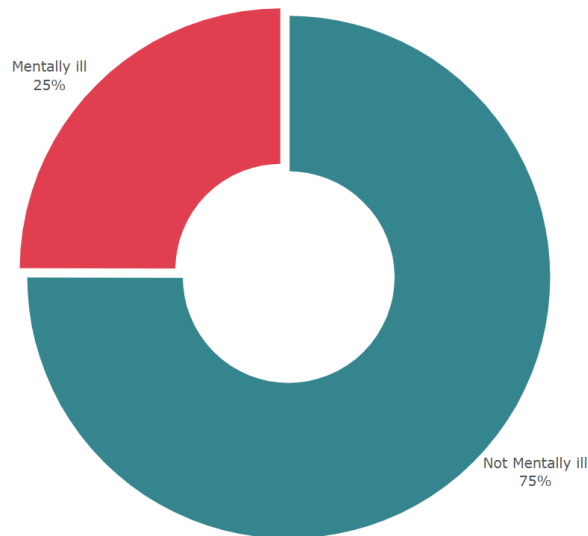
```
fig = px.pie(
    df_killed_mental_illness,
    values='Total_Death',
    names=['Not Mentally ill', 'Mentally ill'],
    hole=0.4,
    color='signs_of_mental_illness',
    color_discrete_map=mentally_ill,
)

# Hover template
fig.update_traces(
    textposition='outside',
    textinfo='percent+label',
    textfont=dict(size=15, color='#555555'),
    marker=dict(line=dict(color='white', width=2)),
    pull=[0.02, 0.02, 0.02, 0.02, 0.02, 0.02],
    hovertemplate=(
        "<b>{%label}</b><br>"
        "<span style='font-size:10px;'>————</span><br>"
        "<b>Fatalities:</b> {%value:,.0f}<br>"
        "<b>Share of Fatalities:</b> {%percent:.1%}<br>"
        "<extra></extra>"
    )
)

# Layout Formatting
fig.update_layout(
    plot_bgcolor='white',
    paper_bgcolor='white',
    title=dict(
        text="<b>Percentage of People Killed by Police with Mental Illness</b><br>",
        font=dict(size=22, color='#333333'),
        x=0.5, y=0.95
    ),
    showlegend=False,
    margin=dict(t=100, b=40, l=40, r=40)
)

fig.show()
```


Percentage of People Killed by Police with Mental Illness



Graph Link: [Percentage of People Killed by Police with Mental Illness](#)

The chart shows that about one in four people who died as a result of police violence between 2015 and 2017 exhibited signs of mental illness; this amounts to more than 630 cases in the dataset—a strikingly high proportion that organizations such as the Treatment Advocacy Center and The Washington Post have repeatedly highlighted.

This suggests that police officers in the United States frequently find themselves in situations that resemble psychiatric crises rather than purely criminal threats.

Many researchers argue that limited access to mental health care, inadequate crisis intervention training, and the lack of specialized emergency teams contribute significantly to these fatal outcomes, particularly during unpredictable encounters with individuals suffering from bipolar disorder, severe depression, or suicidal behavior.

Analysis of Most Dangerous Cities by Police Killings

In this section, we are going to shift our analysis from nationwide patterns to the city level to determine where police killings were most concentrated between 2015 and 2017.

Examining individual cities is important because police practices, crime rates, socioeconomic inequality, population density, gang activity, and local law policies can vary significantly across urban areas, often resulting in very different patterns of fatal incidents, even within the same state.

By analyzing the ethnic composition and total number of deaths in major cities, we aim to better understand in which urban environments fatal police encounters occurred most frequently and whether certain population groups were disproportionately affected by these incidents.

Forming the DataFrames:

After removing entries with unspecified ethnicity, the dataset is grouped by city to calculate the total number of police killings and the top 20 cities with the highest death tolls are identified, and then a second grouping by both city and ethnicity is performed to measure the demographic distribution and calculate the percentage of each ethnic group within each city using vectorized transform() operations.

```
df_top_cities = df_fatalities.copy()

# Filtering for defined races
df_top_cities = df_top_cities.query("race != 'Other'")

# Grouping by city and plotting total killings
df_city_counts =
    df_top_cities.groupby('city').size().reset_index(name='total_killings')

# top 20 cities:
top_cities = df_city_counts
    .sort_values(by='total_killings', ascending=False).head(20)['city']

df_top_cities = df_top_cities[df_top_cities['city'].isin(top_cities)]

# Grouping by city again and plotting also race this time
df_top_cities = df_top_cities
    .groupby(['city', 'State_Full', 'race']).size().reset_index(name='killings')

df_top_cities['city_total'] =
    df_top_cities.groupby('city')['killings'].transform('sum')

# calculating the share of that city
df_top_cities['city_race_pct'] =
    (df_top_cities['killings'] / df_top_cities['city_total']) * 100
```

New York City Fixing:

Since New York City is divided into several boroughs in the original dataset, we manually combine entries at the borough level to a single category labeled “New York City” to enable a more realistic and easily interpretable comparison of the cities.

```
nyc_boroughs = ['Bronx', 'Brooklyn', 'Manhattan', 'Queens', 'Staten Island', 'New York']  
df_top_cities['city'] = df_top_cities['city'].replace(nyc_boroughs, 'New York City')
```

Creating Stacked Bar Graph

We use a horizontal stacked bar chart to compare both the total number of deaths resulting from police encounters and the ethnic composition of the victims in the most affected U.S. cities at the same time.

This allows for a clearer visual comparison between the demographic distribution and the frequency of fatal incidents in these cities.

code:

```
race_colors = {  
    'White': '#2A3F54', 'Black': '#1ABB9C', 'Hispanic': '#E74C3C',  
    'Asian': '#F39C12', 'Native American': '#9B59B6'  
}  
  
fig = px.bar(  
    df_top_cities,  
    x='killings',  
    y='city',  
    orientation='h',  
    color='race',  
    color_discrete_map=race_colors,  
    # Passing the calculated hover data:  
    custom_data=['State_Full', 'killings', 'city_race_pct']  
)  
  
fig.update_traces(  
    marker_line_width=0,  
    hovertemplate=(  
        "<b>{y}</b>, {customdata[0]}</b><br>"  
        "<span style='font-size:10px;'>—————</span><br>"  
        "<b>Demographic:</b> {data.name}<br>"  
    )  
)
```

```

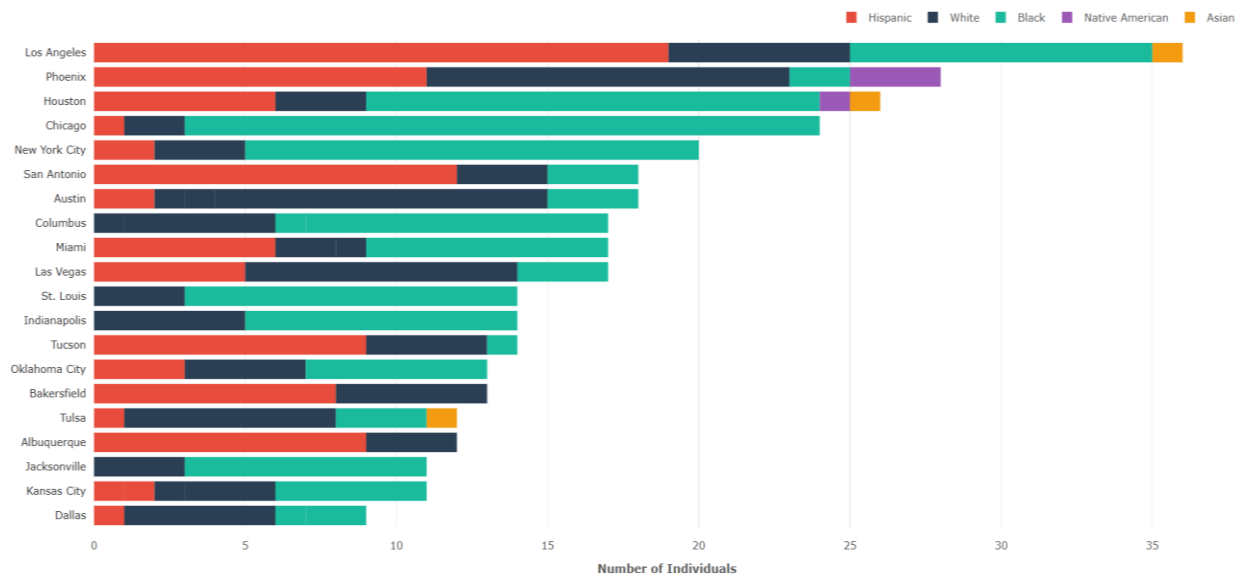
        "<b>Individuals:</b> %<customdata[1]><br>"
        "<b>Share of City Total:</b> %<customdata[2]>:.1f}%<br>"
        "<extra></extra>"
    ),
    hoverlabel=dict(font_size=15)
)

fig.update_layout(
    plot_bgcolor='white',
    paper_bgcolor='white',
    title=dict(
        text='<b>Top Cities by Police Fatalities</b><br>'
        '<span style="font-size: 14px; color: gray;">'
        'Demographic breakdown of individuals within the most affected'
        'cities.</span>',
        font=dict(family="Arial, sans-serif", size=20, color='#333333'),
        x=0.05, y=0.95
    ),
    xaxis=dict(title=dict(text="<b>Number of Individuals</b>",
        font=dict(size=13, color='#555555'), standoff=15),
        tickfont=dict(size=12, color='#555555'),
        showgrid=True,
        gridcolor='#EAEAEA', # Vertical subtle grids
        zeroline=False,
        # Padding:
        ticks = "outside", ticklen = 8, tickcolor = "white"
    ),
    yaxis=dict(
        title="",
        categoryorder='total ascending',
        tickfont=dict(size=12, color='#333333'),
        showgrid=False,
        # Padding:
        ticks="outside", ticklen=6, tickcolor="white"
    ),
    legend=dict(
        title="",
        orientation="h",
        yanchor="bottom", xanchor="right",
        y=1.02, x=1,
        font=dict(size=12, color='#555555')
    ),
    margin=dict(t=110, l=120, r=40, b=60), # Extra left margin for long city names
    height=700
)
fig.show()

```

Top Cities by Police Fatalities

Demographic breakdown of individuals within the most affected cities.



Interactive Graph Link: [Most Dangerous Cities by Police Fatalities](#)

The graph shows that police killings were heavily concentrated in a few major metropolitan areas such as Los Angeles, Phoenix, Houston, Chicago, and New York City; however, the demographic distribution of those killed varied significantly from city to city and often reflected broader historical patterns such as isolation, areas of poverty, migration, and the intensity of police activity.

For example, Black people accounted for nearly 88% of fatalities in Chicago and 75% in New York City, while Hispanics made up the majority of fatalities in cities such as San Antonio, Albuquerque and Tucson. This illustrates how local demographic patterns and neighborhood-level policing strategies shape the patterns of fatalities.

Several of these cities are already at the center of nationwide debates on police violence since years, particularly Chicago, Los Angeles, Houston, and New York City, where controversies surrounding “stop-and-frisk” practices, police operations in areas affected by gang crime, allegations of racial profiling, and patrols in high-crime neighborhoods sparked significant public criticism and large-scale protests.

Analysis of Geographic Distribution of Police Fatalities

In this section, we are going to expand our analysis from the city level to the state level to identify which U.S. states recorded the highest number of police killings between 2015 and 2017, and to examine whether these patterns are linked to broader socioeconomic challenges such as poverty.

The geographic analysis of these deaths is important because the intensity of policing, gun ownership, population density, violent crime rates, urbanization, and social inequalities vary significantly across the country at the state level and can influence the frequency of fatal police interventions.

By comparing the total number of deaths with each state's share of the population, we aim to better understand whether certain states are disproportionately represented in police killings and whether these regions also align with states previously identified as economically disadvantaged or socially vulnerable.

Forming the DataFrame:

After grouping all fatality records by state and counting the total number of incidents, the results are merged with nationwide population statistics and then calculated each state's percentage share of the total number of killings by U.S. police, using vectorized arithmetic operations for efficient large-scale computation

```
df_fatalities_by_state = df_fatalities.groupby(['state', 'State_Full'])
                                .size().reset_index(name='killings')

df_fatalities_by_state = df_fatalities_by_state
                                .merge(df_state_pop, on='State_Full', how='left')

total_us_fatalities = df_fatalities_by_state['killings'].sum()

df_fatalities_by_state['killing_by_police_usa_share_pct'] =
    (df_fatalities_by_state['killings'] / total_us_fatalities) * 100
```

Creating choropleth for the Visualization Statewide Police Fatalities

We use a choropleth map to illustrate the frequency of police killings to more clearly identify regional patterns and states particularly affected by this issue through color variations and population-adjusted contextual information.

code:

```
fatality_colorscale = [
    [0.00, "#F6E3B4"], [0.08, "#EFBA6B"], [0.18, "#E57827"], [0.32, "#B73123"],
    [0.40, "#BF2F4E"], [0.50, "#AB1D56"], [0.60, "#7B0F45"], [0.70, "#5C0B3A"],
    [0.80, "#870B26"], [1.00, "#240014"]
]

world_map = px.choropleth(
    df_fatalities_by_state,
    locations='state',
    locationmode='USA-states',
    color='killings',
    scope="usa",
    color_continuous_scale=fatality_colorscale,
    custom_data=['State_Full', 'killings', 'total_state_population',
'killing_by_police_usa_share_pct']
)

world_map.update_traces(
    hovertemplate=(
        "<b>{%customdata[0]}</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>Individuals:</b> {%customdata[1]:,.0f}<br>"
        "<b>Share of US Total:</b> {%customdata[3]:.1f}%<br>"
        "<b>State Population:</b> {%customdata[2]:,.0f}<br>"
        "<extra></extra>"
    ),
    marker_line_width=0.5,
    marker_line_color='white',
    hoverlabel=dict(bgcolor="#831e70", font_size=13),
)

world_map.update_layout(
    title=dict(
        text='<b>Police Fatalities by US State</b><br>'
        '<span style="font-size: 14px; color: gray;">'
        'Total individuals per state and their share of the national'
        'total.</span>',
        font=dict(family="Arial, sans-serif", size=22, color='#831e70'),
        x=0.05, y=0.95
    ),
    geo=dict(
        bgcolor='rgba(0,0,0,0)',
        lakecolor='#FFFFFF', landcolor='#F4F4F4',
        showlakes=True, showcoastlines=False,
        projection_type='albers usa'
    ),
)
```

```

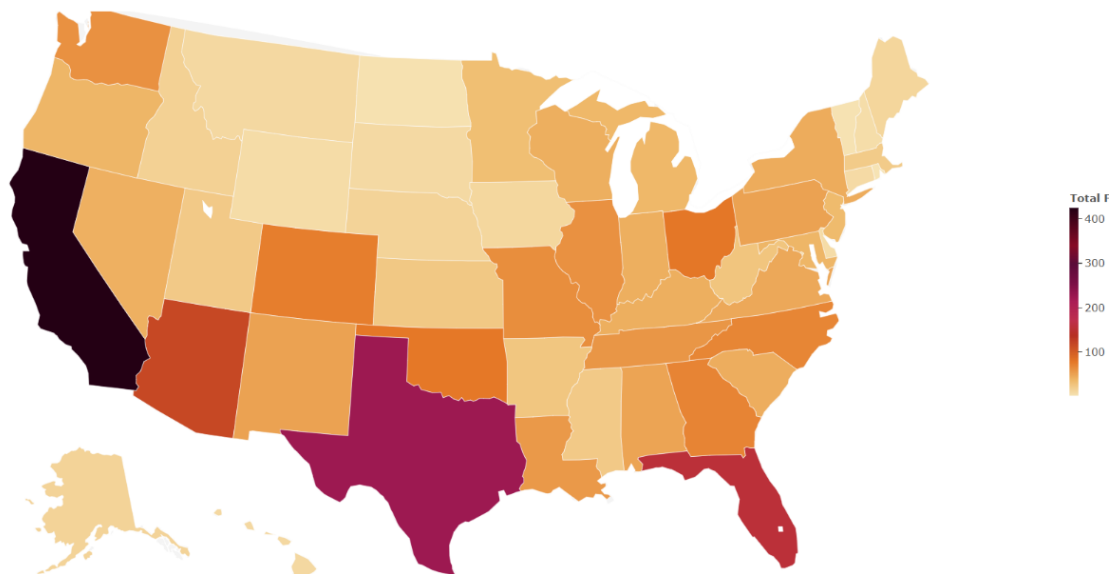
coloraxis_colorbar=dict(
    title=dict(text="<b>Total Fatalities</b>", font=dict(size=12)),
    thicknessmode="pixels", thickness=10,
    lenmode="pixels", len=250,
    yanchor="middle", y=0.5,
    xanchor="left", x=1.02,
    ticks="outside",
    tickfont=dict(color="#555555")
),
plot_bgcolor='white',
paper_bgcolor='white',
margin=dict(l=0, r=0, t=80, b=0)
)

world_map.show()

```

Police Fatalities by US State

Total Individuals per state and their share of the national total.



Interactive Map Link: [Police Fatalities by US State](#)

The figure shows a high geographical concentration of deaths related to police interventions in several large states in the South and West, particularly in California, Texas, Florida, Arizona, and Oklahoma, which together account for a significant proportion of all deaths recorded in the dataset.

California alone accounts for 424 deaths, nearly 17% of all police killings nationwide, while Texas accounts for nearly 9% a pattern likely influenced not only by these states' very large populations but also by factors such as high gun ownership rates, a high concentration of crime in metropolitan areas, the dynamics of border security, gang crime, aggressive policing practices, and general socioeconomic inequality.

At the same time, however, the graph also shows that population size alone cannot fully explain these differences, as some smaller or medium-sized states—such as Oklahoma, New Mexico, Arizona, and Louisiana—have disproportionately high death rates relative to their population size, while populous states in the Northeast—such as New York, Massachusetts, and New Jersey—have comparatively lower numbers.

Interestingly, many of the states with high rates of police killings—including Louisiana, Alabama, Mississippi, New Mexico, and Oklahoma—were previously identified in this analysis as states with relatively high poverty rates and lower median incomes. This supports broader criminological research suggesting that economic instability, concentrated poverty, limited educational opportunities, and a higher risk of violent crime can increase the likelihood of dangerous police encounters and deadly confrontations.

Analysis of the Relationship Between Income and Fatality Rates

In this section, we are going to examine whether economic conditions might be linked to the frequency of fatal police interventions in individual U.S. states by comparing median household income with fatality rates during police interventions.

Rather than focusing solely on raw fatality numbers, this analysis examines the broader socioeconomic context of each state to determine whether lower-income regions are disproportionately affected by fatal police interventions.

By highlighting potential links between economic inequality and police violence, we aim to better understand how poverty, social instability, crime risk, and community resources may influence patterns of fatal police encounters in the United States.

Forming the DataFrame:

To perform this analysis, the previously created datasets on income and on fatality rates are combined using the function `.merge()`, so that economic indicators and indicators of deaths could be analyzed together in a single structured DataFrame.

We then apply vectorized mathematical operations to calculate a standardized metric: Deaths per 1 million residents. This normalizes the data relative to population size and enables meaningful comparisons between states of vastly different sizes.

Without this type of normalization, populous states like California or Texas would dominate the analysis solely due to their size (as we have seen in previous analysis), whereas the per-capita approach provides a more accurate picture of the relative frequency of fatal police interventions.

```
df_income_vs_killing = df_income_by_state
                        .merge(df_fatalities_by_state, on='State_Full', how='inner')

# Calculating Per Capita Rate (Killings per 1 Million people)
df_income_vs_killing['killings_per_1m'] =
    (df_income_vs_killing['killings']
     / df_income_vs_killing['total_state_population_x'])
    * 1000000
```

Creating the Scatter Plot with Statistical Trendline

We use a scatter plot in combination with a regression line based on the ordinary least squares (OLS) method to visually assess whether there is a statistical correlation between median income and the number of fatalities in each state.

Each point represents a single state, while the regression line summarizes the overall trend. This makes it easier to see whether states with higher incomes generally have lower per capita rates of fatal police interventions compared to economically disadvantaged states.

code:

```
# Scatter Plot with Ordinary Least Squares (OLS) trendline
fig = px.scatter(
    df_income_vs_killing,
    x='Median_Income',
    y='killings_per_1m',
```

```

    hover_name='State_Full',
    trendline='ols',
    trendline_color_override='#c0392b',
    custom_data=['total_state_population_x', 'killings', 'Median_Income']
)

fig.update_traces(
    marker=dict(
        size=20,
        color='#005691',
        opacity=0.6,
        line=dict(width=1, color='rgba(50, 50, 50, 0.8)') # borders around dots
    ),
    hovertemplate=(
        "<b>{%hovertext}</b><br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>Median Income:</b> ${{customdata[2]:.0f}}<br>"
        "<b>Rate per 1M Residents:</b> {{y:.1f}}<br>"
        "<span style='font-size:10px;'>—————</span><br>"
        "<b>Individuals Killed by Police:</b> {{customdata[1]}}<br>"
        "<b>Total Population:</b> {{customdata[0]:.0f}}<br>"
        "<extra></extra>"
    ),
    selector=dict(mode='markers') # white borders only apply to the dots.
)

# Customizing the trendline
trendline_trace = fig.data[1]
trendline_trace.line.color = '#DC3545'
trendline_trace.line.dash = 'dash'
trendline_trace.line.width = 5

# Layout Adjustments
fig.update_layout(
    title=dict(
        text='<b>Income vs. Police Fatalities by State</b><br>'
        '<span style="font-size: 14px; color: gray;">Analyzing the  
relationship between median household income and fatalities per 1M  
residents.</span>',
        font=dict(family="Arial, sans-serif", size=20, color="#333333"),
        x=0.05, y=0.95
    ),
    xaxis=dict(
        title=dict(text="<b>Median Household Income</b>", font=dict(size=13,
            color="#555555"), standoff=15),
        tickfont=dict(size=12, color="#555555"),
        tickprefix="$", tickformat=",.0f",
        showgrid=True, gridcolor='#EAEAEA',
        zeroline=False
    )
)

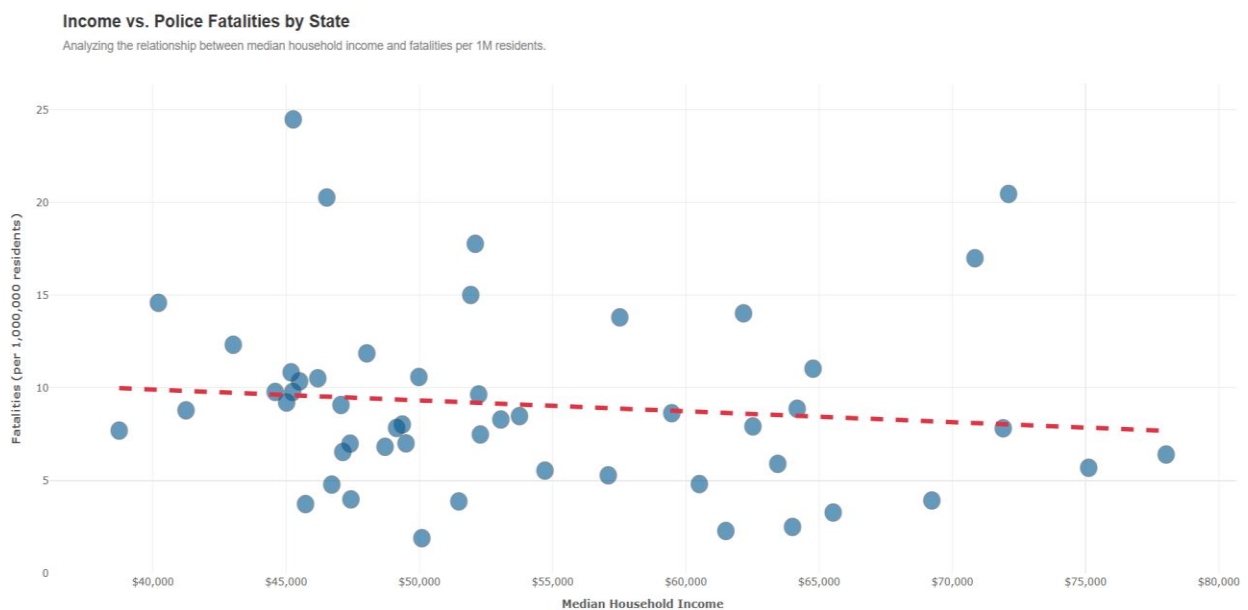
```

```

),
yaxis=dict(
    title=dict(text="<b>Fatalities (per 1,000,000 residents)</b>",
        font=dict(size=13, color="#555555", standoff=15),
        tickfont=dict(size=12, color="#555555"),
        showgrid=True, gridcolor='#EAEAEA',
        zeroline=False
    ),
    plot_bgcolor='white',
    paper_bgcolor='white',
    margin=dict(l=60, r=40, t=100, b=60),
    hoverlabel=dict(font_size=13, font_family="Arial", bordercolor="#EAEAEA")
)

fig.show()

```



Interactive Graph Link: [Income vs. Police Fatalities by State](#)

The scatter plot suggests that there is only a very weak correlation between median household income and the number of fatal police interventions in the U.S. states.

Although the OLS regression line slopes downward slightly—suggesting that states with higher incomes tend to have slightly fewer fatal police incidents—the extremely low R^2 value of 0.013932 suggests that median income explains only about 1.4% of the variation in fatal police interventions, meaning that income alone is not a strong predictor of fatal police interventions.

This is an important finding, as it shows that police violence is subject to far more complex structural factors such as gun ownership rates, police practices, crime rates, drug abuse, racial inequality, mental health crises, and state-level law policies.

Several states clearly support this interpretation, as they represent statistical outliers. New Mexico, Oklahoma, Alaska, Arizona, and the District of Columbia have some of the highest death rates per million residents despite very different economic conditions, suggesting that geographic factors, policing practices, and demographic factors may play a greater role than income alone.

Alaska is particularly noteworthy: Although it appeared relatively moderate on the choropleth map due to its low total number of deaths (15), the small population means that the per capita death rate rises dramatically to over 20 deaths per million residents—one of the highest in the United States; the situation is similar in Washington, D.C., which is also likely due to its relatively small population.

In contrast, California initially appeared to be extremely dangerous due to its very high absolute number of deaths (424); however, when normalized by population, the rate drops to about 11 deaths per million residents—only slightly above the expected trend value of about 8.5 for this income level—which illustrates why per capita normalization is essential for fair comparisons between states.

The graph also shows some notable outliers: states such as New York, New Jersey, Massachusetts, and Rhode Island have relatively high incomes and lower mortality rates than the OLS trend line would suggest, while states such as New Mexico and Oklahoma have disproportionately high mortality rates despite only moderate-income levels.

Overall, the analysis suggests that while poorer states tend to have slightly higher fatality rates following police encounters, this correlation is far too weak to conclude that income alone is a significant causal factor.

The key insight from the graph is therefore not the presence of a strong economy, but rather the observation that fatal police interventions cannot be adequately explained by income statistics alone but must instead be understood as a complex social and institutional problem.

Conclusion

This analysis examined fatal incidents involving police in the United States between 2015 and 2017 from various perspectives, including ethnicity, gender, age, mental health conditions, geographic location, income level, and population-adjusted death rates, in order to better understand the patterns underlying fatal police interventions. By combining statistical and visual analysis, the study demonstrated that fatal police encounters are shaped by many interrelated social and institutional factors while also highlighting how demographic composition, regional differences, and socioeconomic conditions can significantly influence the interpretation of raw data.

From a data science perspective, a wide range of analysis and visualization techniques were employed as part of the project to transform raw data sets into meaningful insights. Advanced DataFrame operations based on feature engineering and weighted-value standardization were used to create fair comparisons between different aspects, while interactive visualizations made complex patterns immediately understandable. Special attention was also paid to data preprocessing and contextual interpretation, such as handling missing demographic values and distinguishing between absolute totals and population-adjusted rates to avoid misleading conclusions. Overall, the analysis demonstrates how modern data science methods can combine statistical thinking, programming, visualization design, and critical interpretation to examine sensitive real-world issues in a structured and evidence-based manner.

References:

<https://www.udemy.com/course/100-days-of-code/>

<https://www.washingtonpost.com/graphics/investigations/police-shootings-database/>

<https://www.census.gov/>

https://pandas.pydata.org/docs/user_guide/index.html#user-guide

<https://numpy.org/doc/stable/>

<https://plotly.com/python/getting-started/>

<https://plotly.com/python/horizontal-bar-charts/>

<https://seaborn.pydata.org/generated/seaborn.histplot.html>

<https://plotly.com/python/line-and-scatter/>

<https://plotly.com/python/choropleth-maps/>